SMT-based Attack on Register Transfer Level Locking

Dr. Chandan Karfa[1]

[1]Indian Institute of Technology Guwahati, India

June 30, 2021

Outline

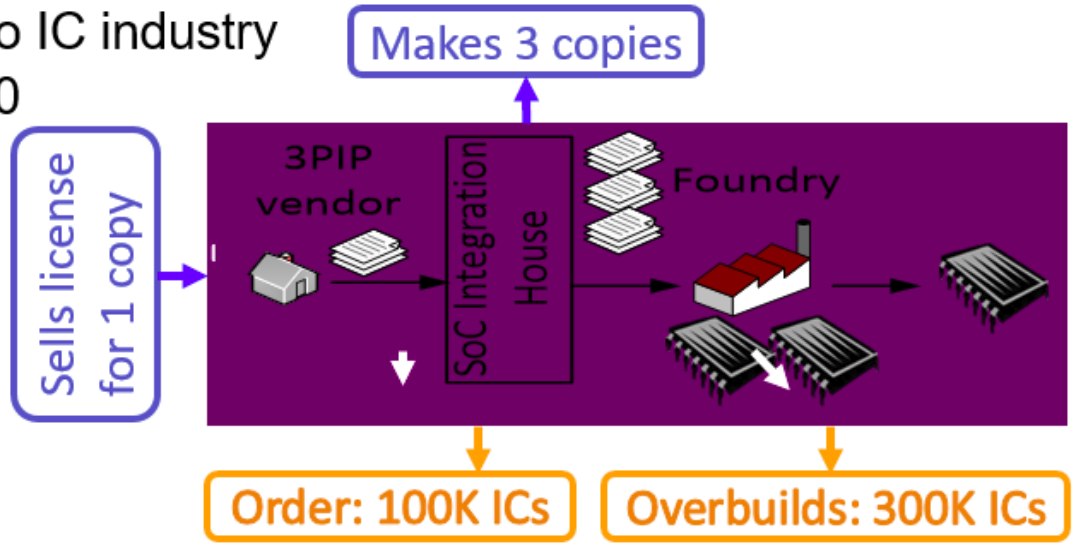
1. Introduction
2. TAO
3. Motivation
4. Overall Approach
5. Tool Flow
6. Experimental Result
7. Conclusion

IC/IP piracy and Overbuilding

- **Steal and claim ownership** of IC and/or illegal use
  - Malicious SoC integration house
  - Malicious foundry
- Real-life impact
  - $4,000,000,000 loss per year to IC industry
  - ARM detected IP piracy in 2000

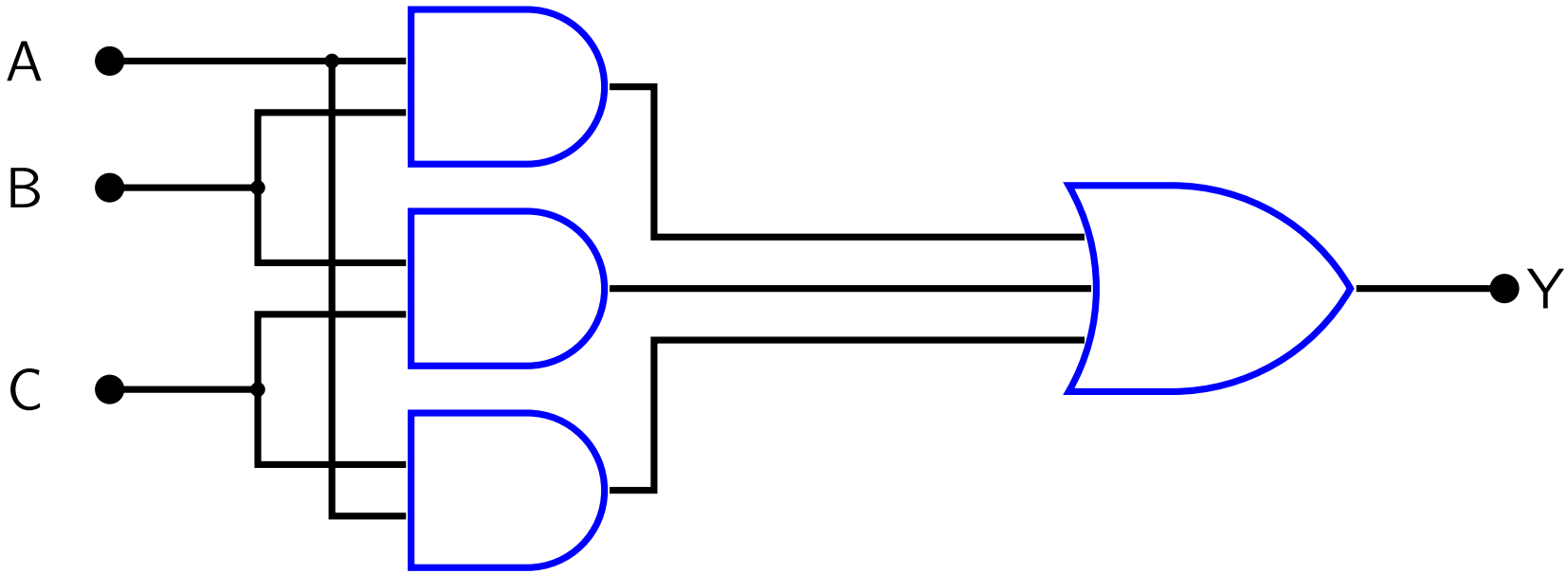
semi
AS SEMICONDUCTOR EQUIPMENT A
LOSES UP TO $4 BILLION ANNUALLY

EE Times
ARM files patent infringement suit against IP startup picoTurbo



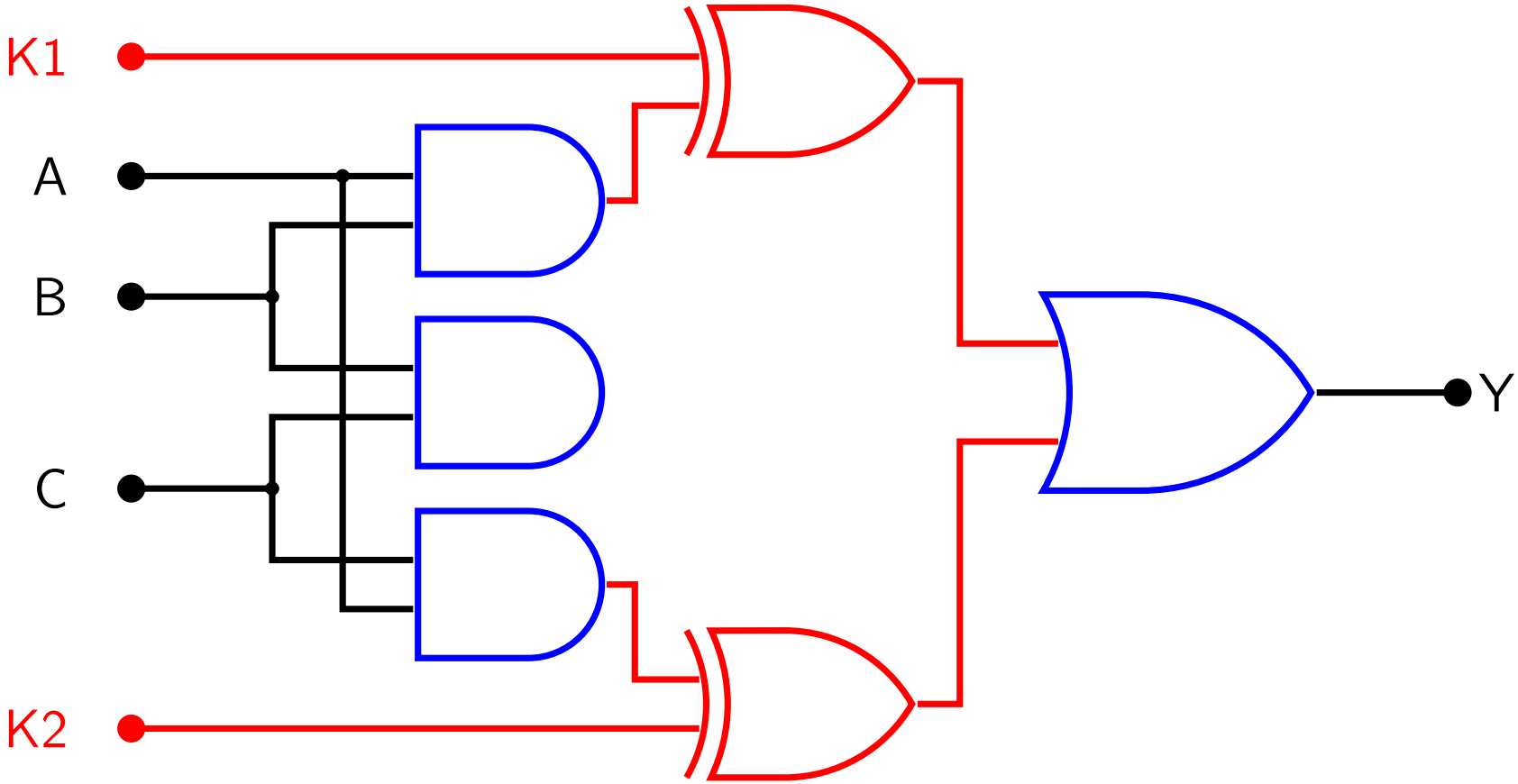
Solution: Logic Locking

- Obfuscate the inner details of the original design
- A secret key can only unlock the chips.

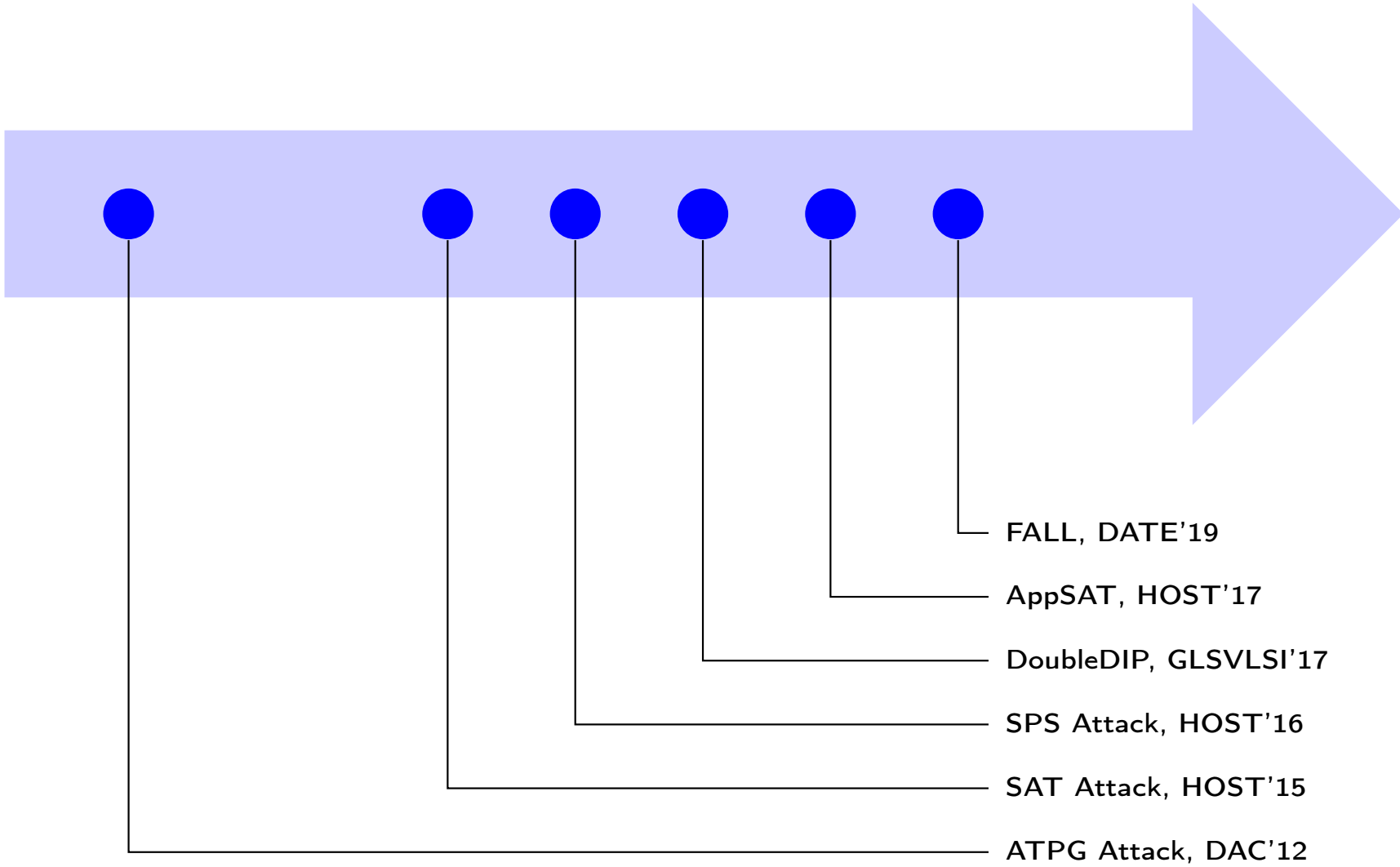


Solution: Logic Locking

- Obfuscate the inner details of the original design
- A secret key can only unlock the chips.

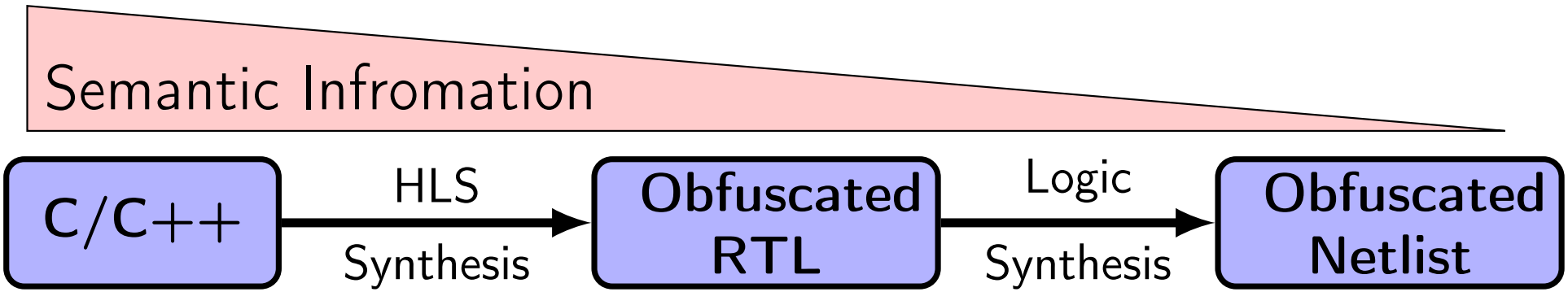


Logic Locking: A Timeline of Attacks



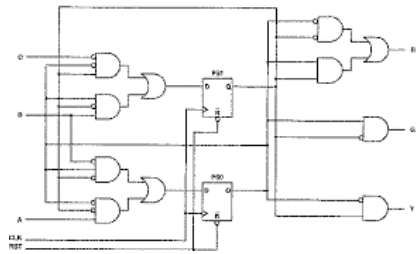
Raising the Abstraction Level

- Secure design at algorithm-level
- Algorithm-level security secures/uses (meaningful) semantic info
- Security Engineer can identify/protect sensitive info




```
for (i=0; i<N; i++)
  c[i] = a[i]+b[i];
   ...
```

```
always @ posedge clk
  a[i] <= b[i] + c[i];
   ...
```



Algorithm-level Obfuscation

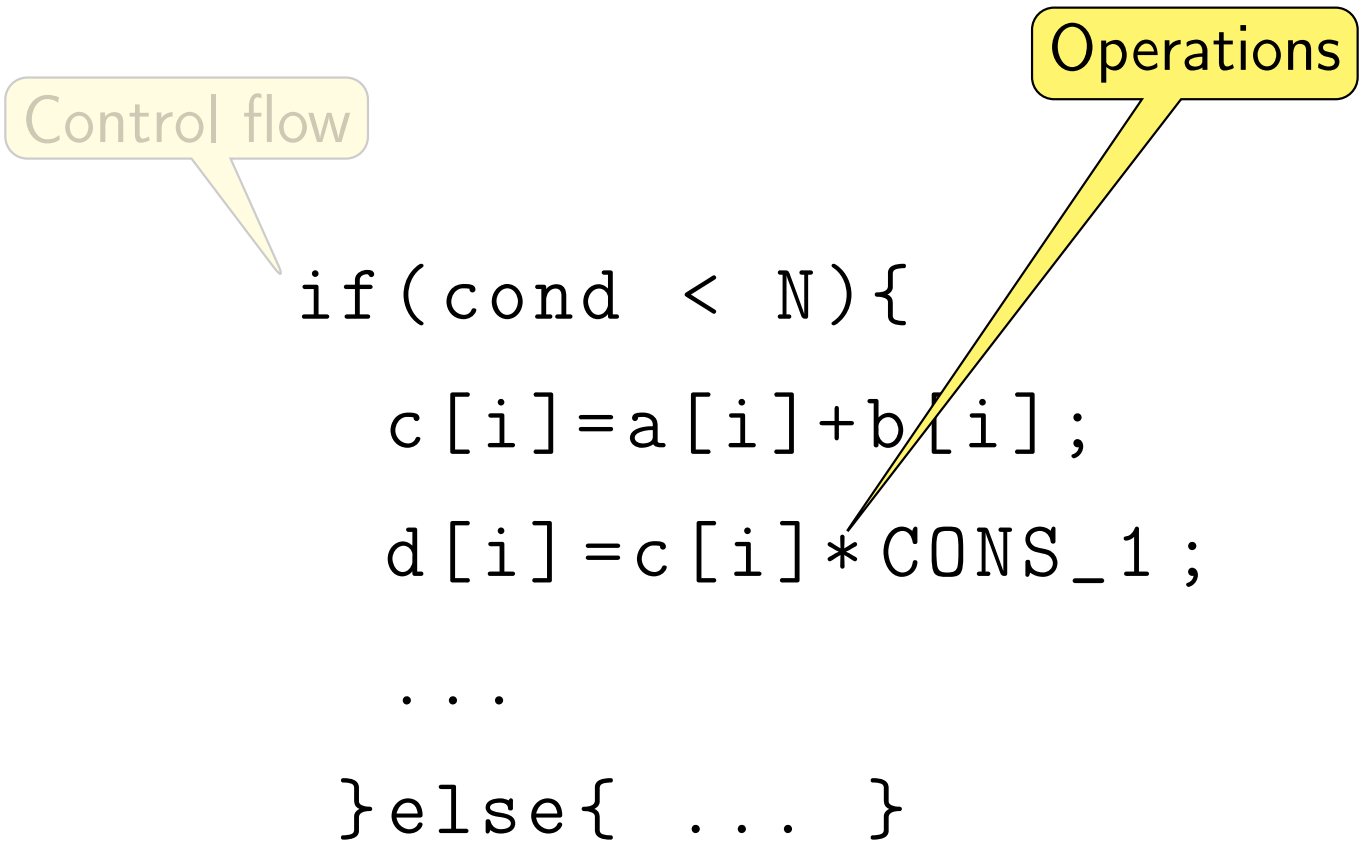
```
if(cond < N){
  c[i]=a[i]+b[i];
  d[i]=c[i]*CONS_1;
  ...
 }else{ ... }
```

Algorithm-level Obfuscation

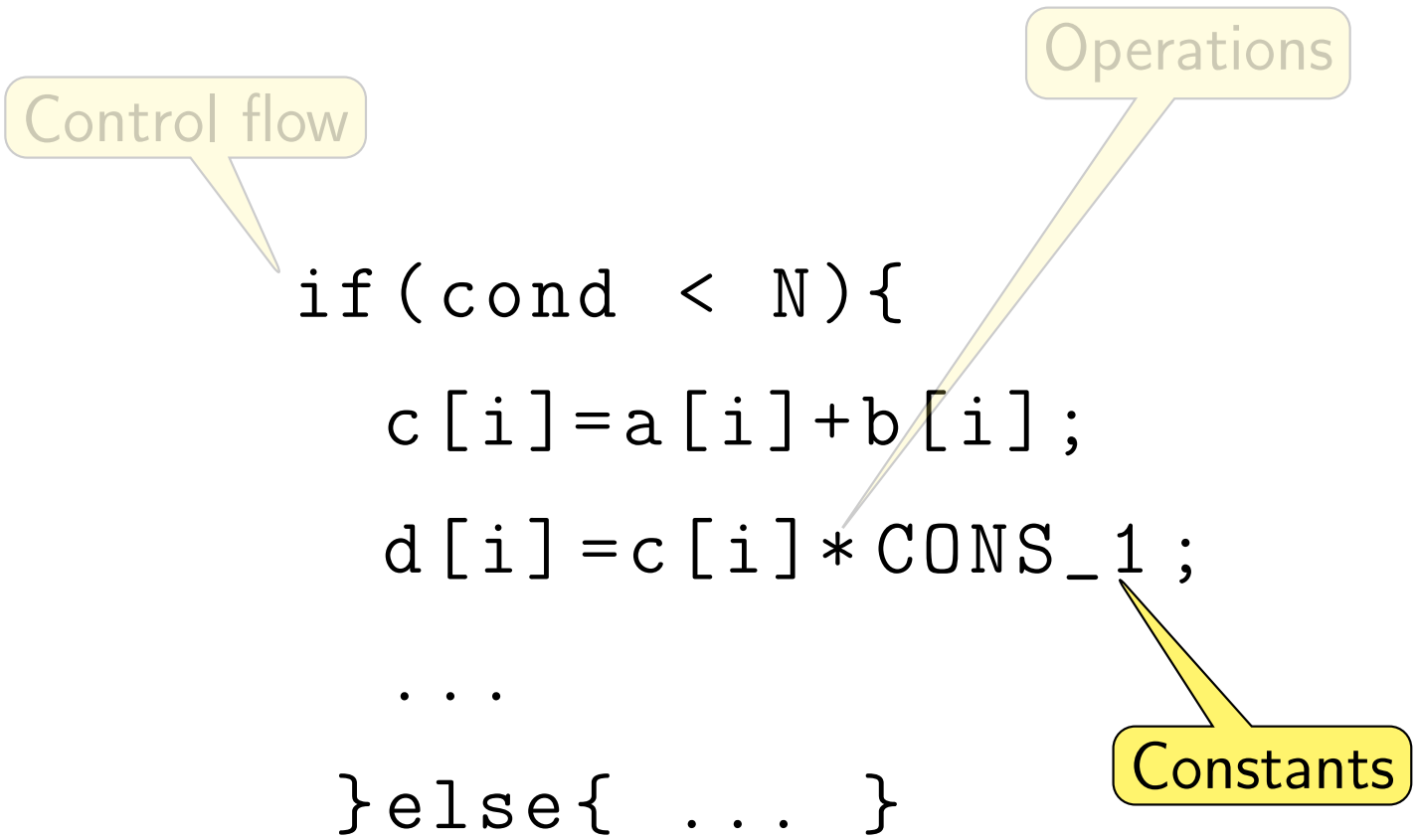
Control flow

```
if(cond < N){
  c[i]=a[i]+b[i];
  d[i]=c[i]*CONS_1;
  ...
}else{ ... }
```

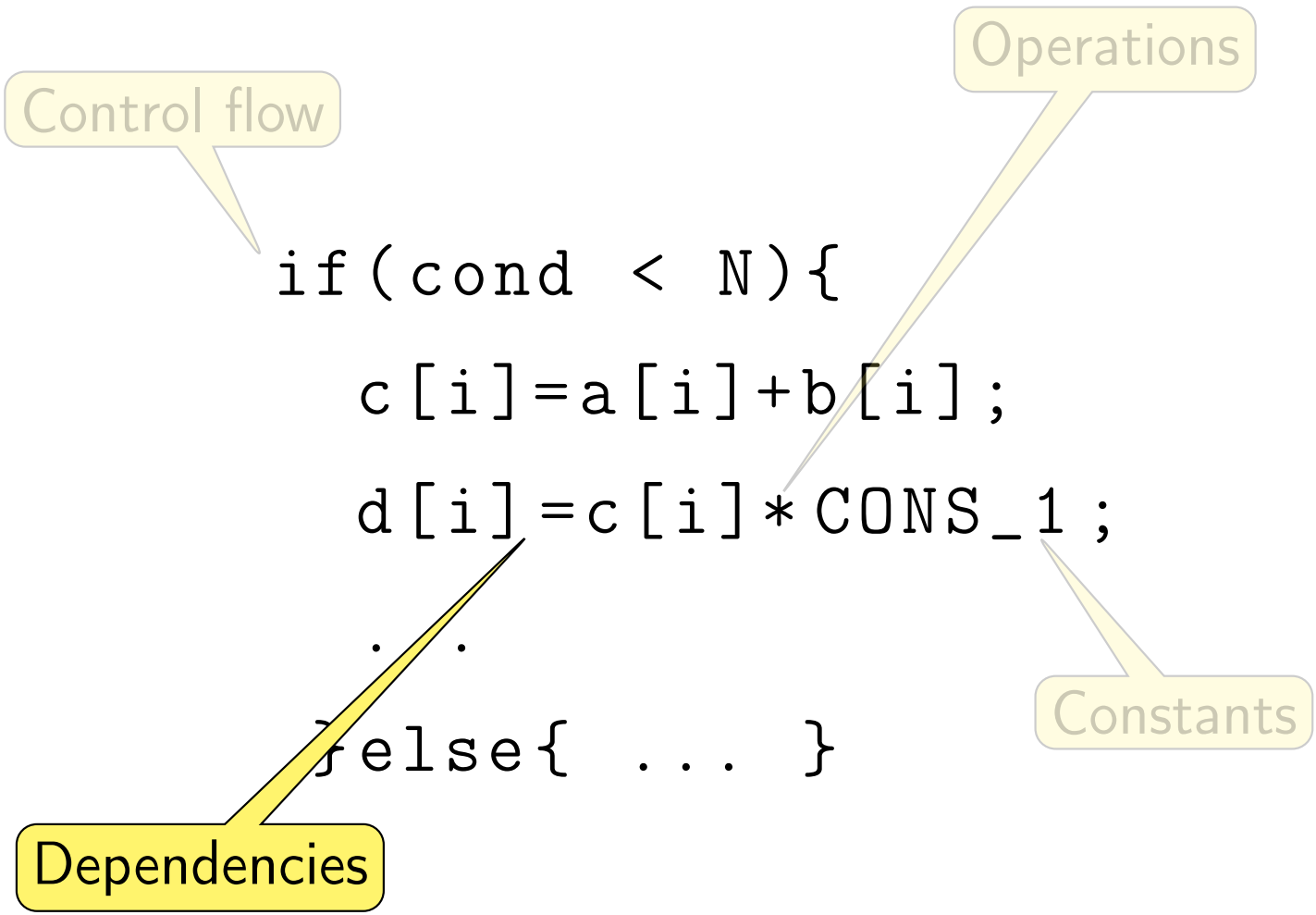
Algorithm-level Obfuscation



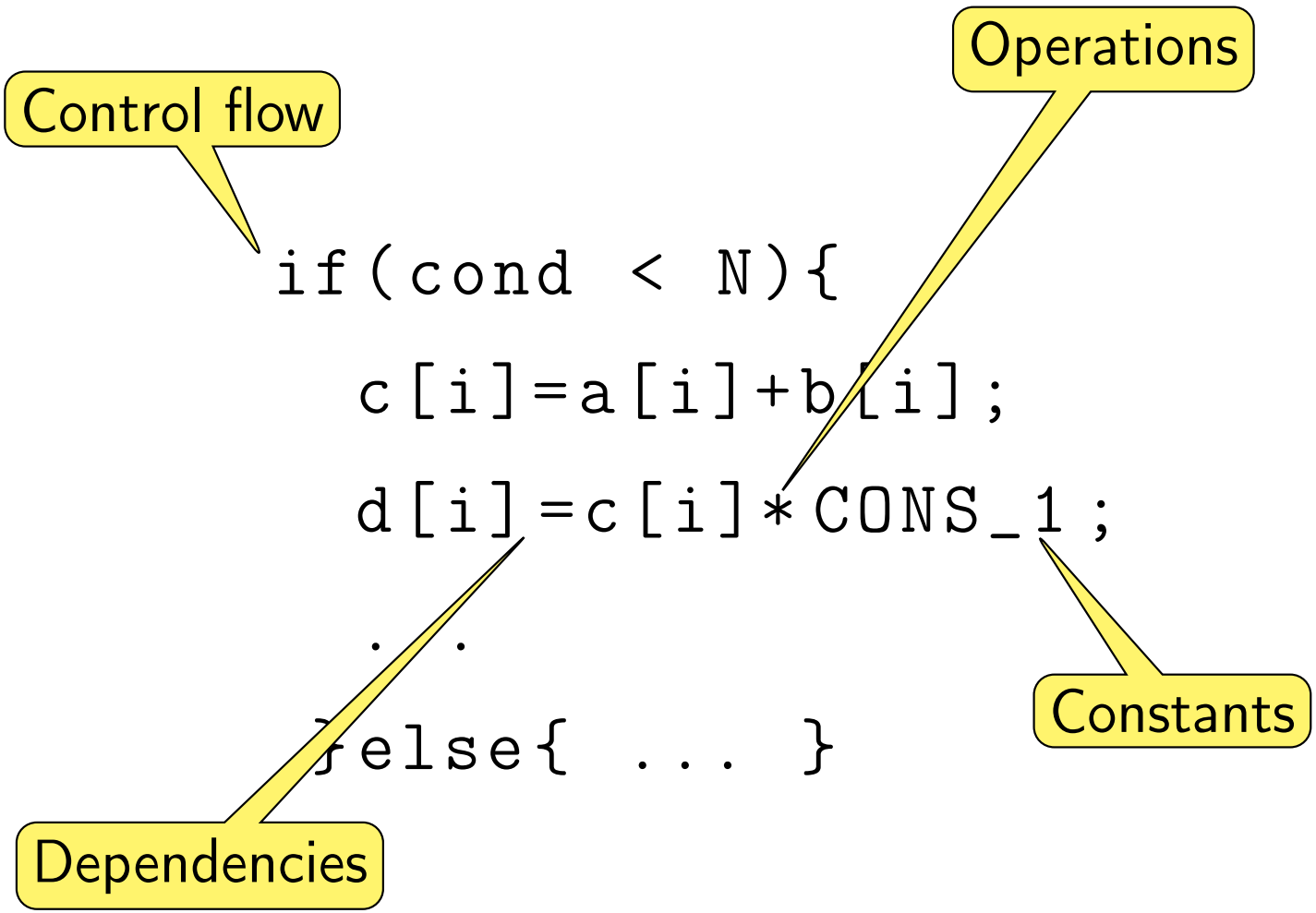
Algorithm-level Obfuscation



Algorithm-level Obfuscation



Algorithm-level Obfuscation



TAO: Techniques for algorithm-level obfuscation

Control-flow Obfuscation

- Control-flow branches describe the evolution of the algorithm

Control condition

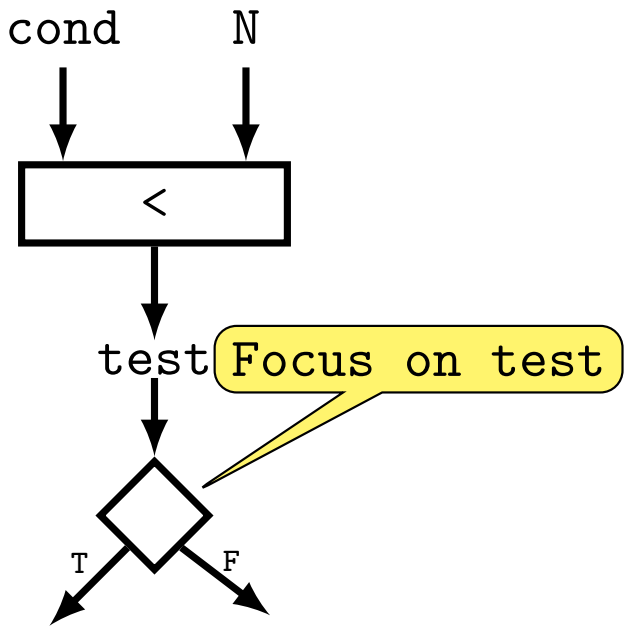
```
if(cond < N){
  c[i]=a[i]+b[i];
  d[i]=c[i]*CONST_1;
  ...
 }else{ ... }
```

Control-flow Obfuscation

- Control-flow branches describe the evolution of the algorithm
- Mask control condition with key bit

Control condition

```
if(cond < N){
  c[i]=a[i]+b[i];
  d[i]=c[i]*CONST_1;
  ...
 }else{ ... }
```

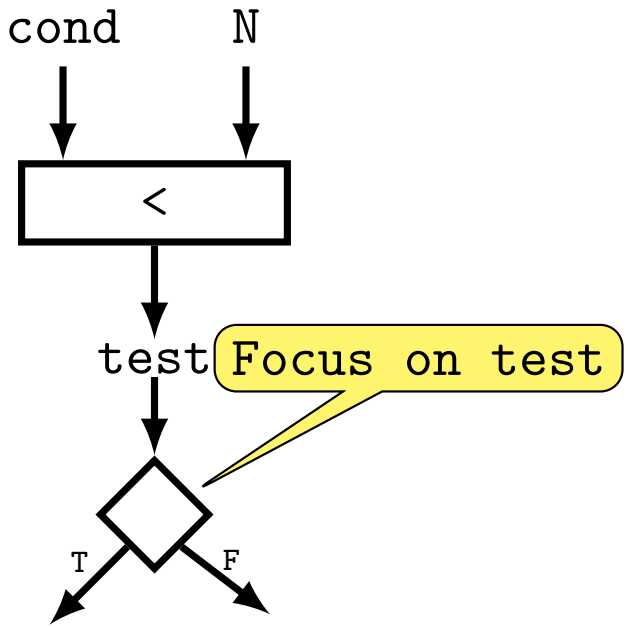


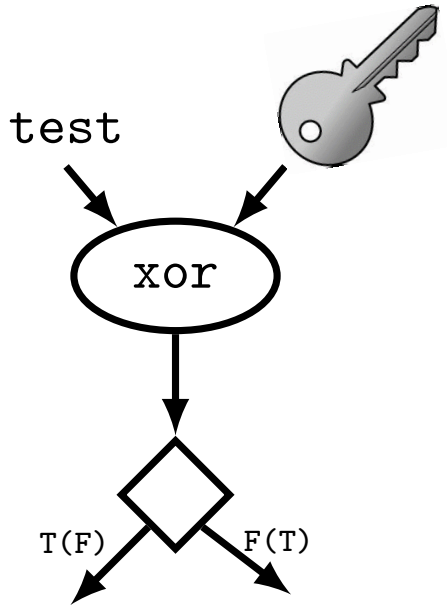
Control-flow Obfuscation

- Control-flow branches describe the evolution of the algorithm
- Mask control condition with key bit
- Correct branch is taken only with correct key
- Reorder Branch: Ensures semantic equivalence + confuse attacker

Control condition

```
if(cond < N){
  c[i]=a[i]+b[i];
  d[i]=c[i]*CONST_1;
  ...
 }else{ ... }
```

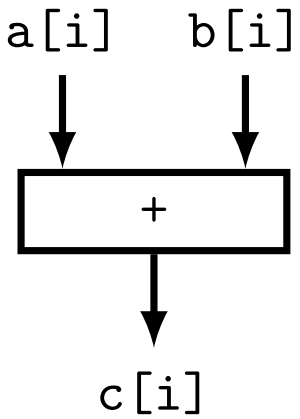




Operation Obfuscation

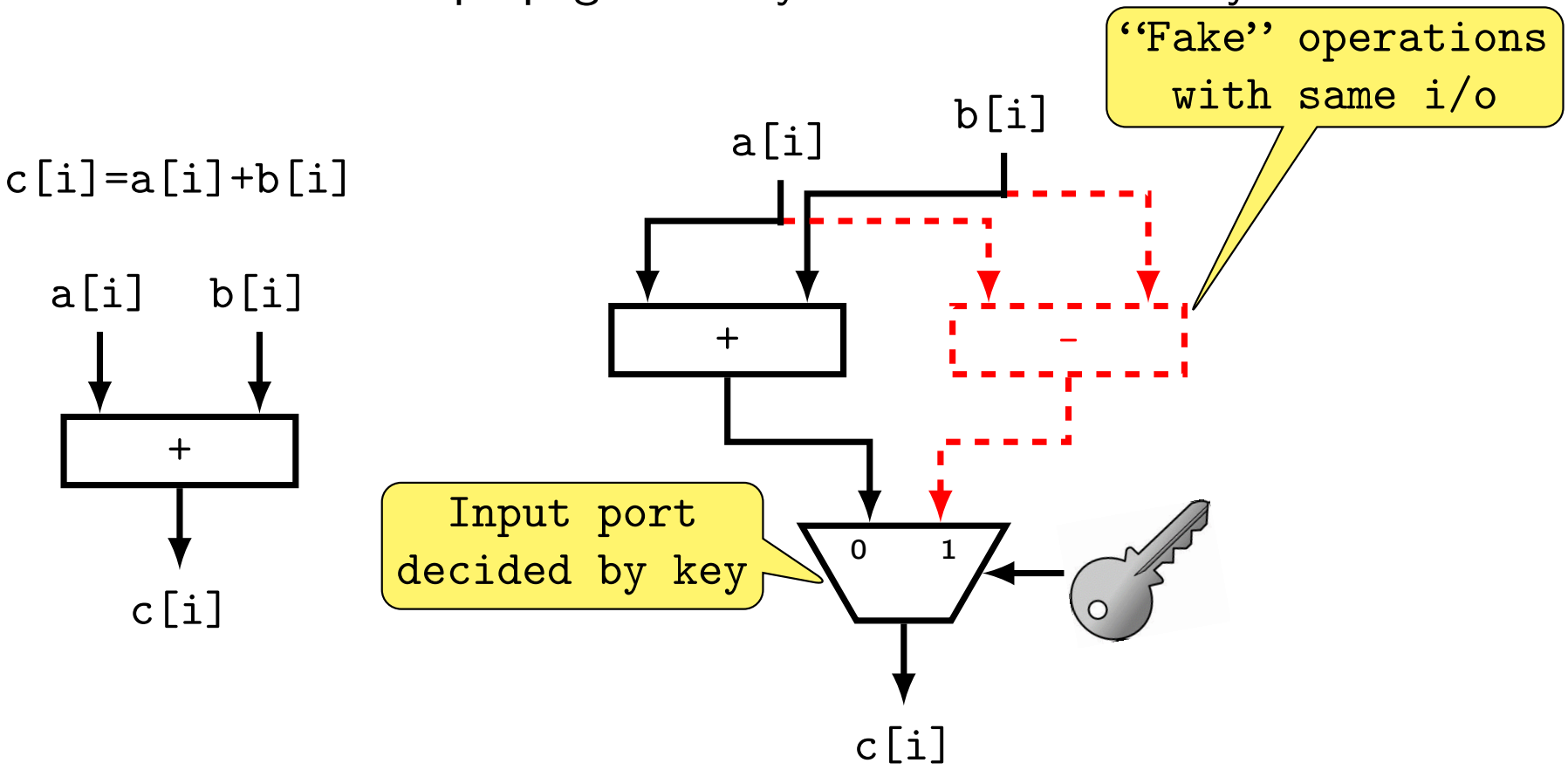
- Gives intelligence on what the algorithm does

```
c[i]=a[i]+b[i]
```



Operation Obfuscation

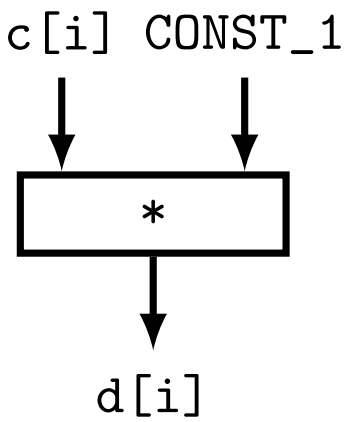
- Gives intelligence on what the algorithm does
- Operator variants can camouflage correct operation
- Correct result is propagated only with the correct key



Constant Obfuscation

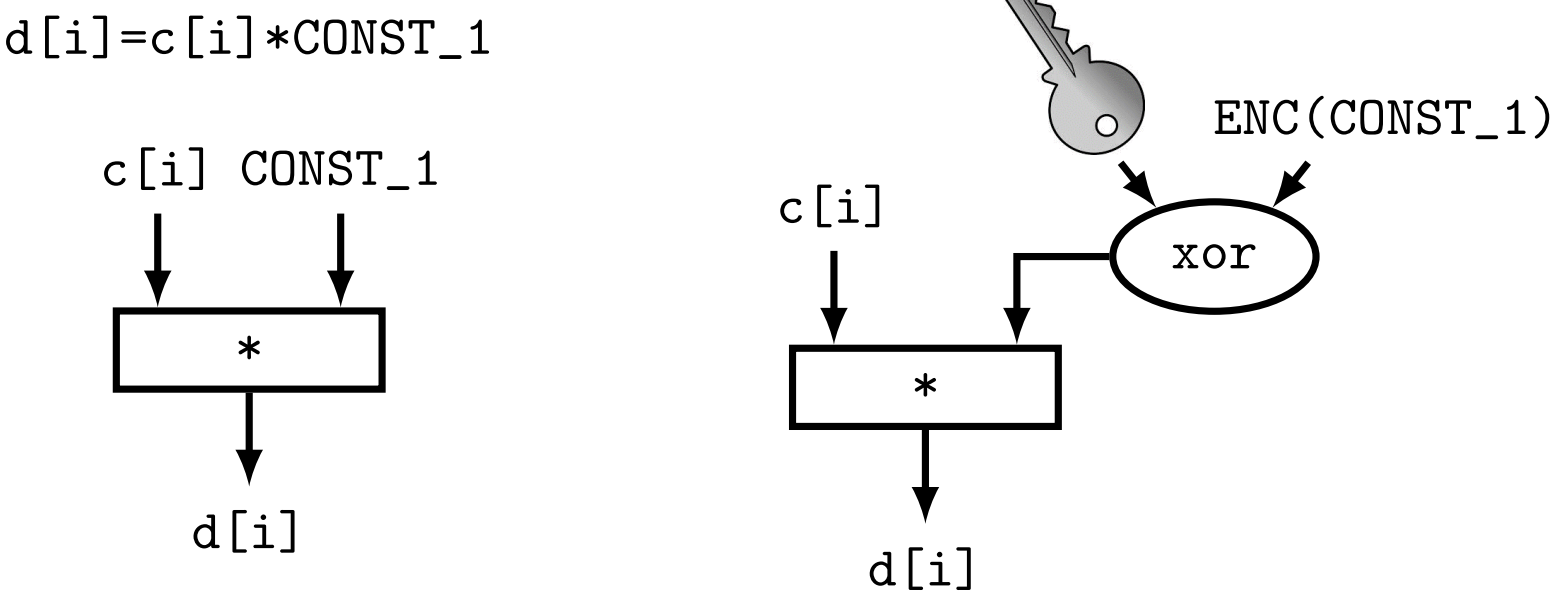
- Constant represents hard-coded values used by algorithm (coefficients, thresholds, ...)

```
d[i]=c[i]*CONST_1
```

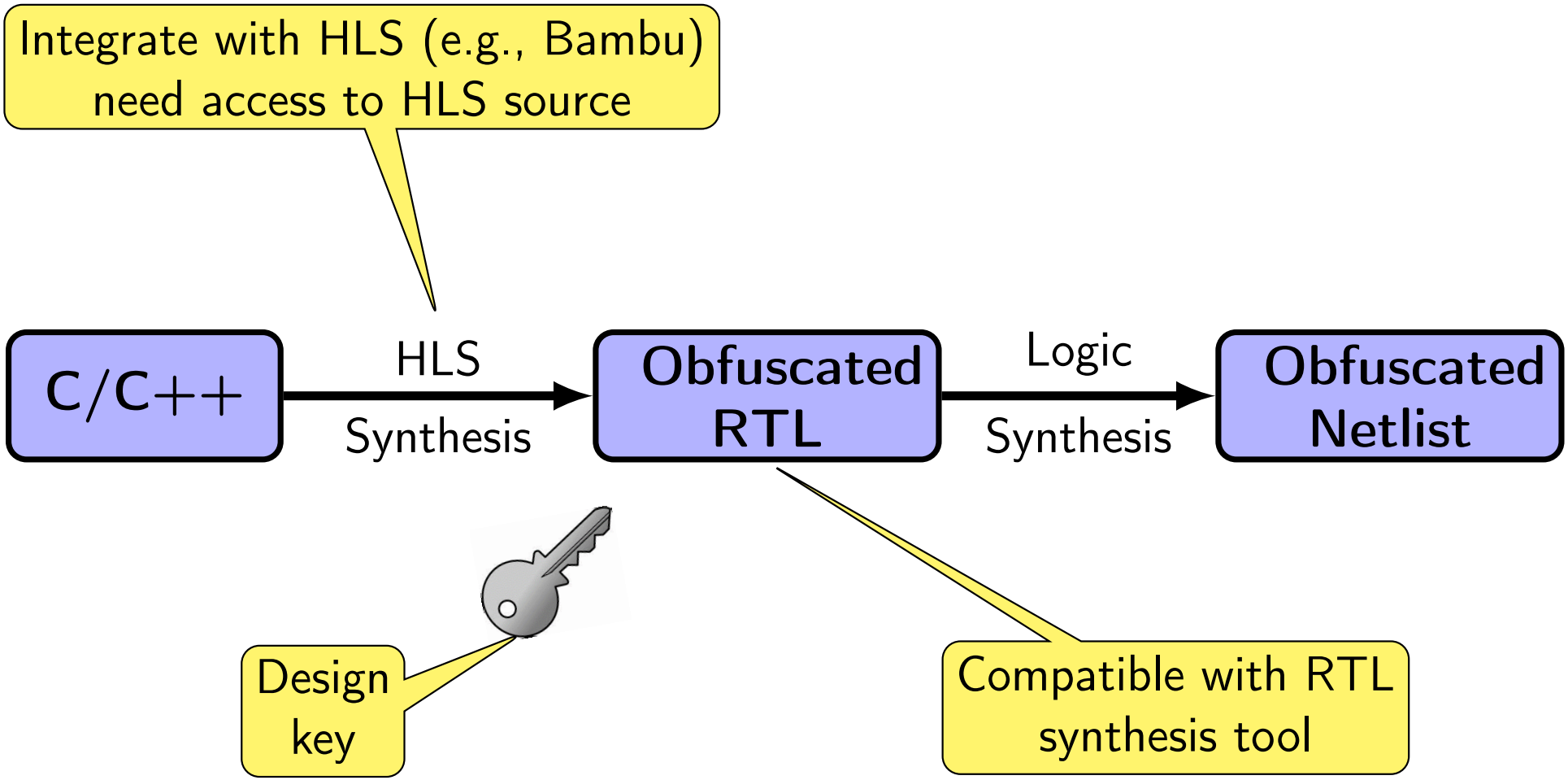


Constant Obfuscation

- Constant represents hard-coded values used by algorithm (coefficients, thresholds, ...)
- A *m*-bit constant is extracted and encoded using *m* working key bits



HLS Obfucation



Semantic Obfuscation: Branches, Dependencies, Operations, Constants

Motivations

- Gate-level SAT attack is not scalable to practical designs with hundreds of thousands of gates and flip-flops.

Motivations

- Gate-level SAT attack is not scalable to practical designs with hundreds of thousands of gates and flip-flops.
- RTL locked netlists are large and consequently less vulnerable to conventional gate-level SAT attacks.

Motivations

- Gate-level SAT attack is not scalable to practical designs with hundreds of thousands of gates and flip-flops.
- RTL locked netlists are large and consequently less vulnerable to conventional gate-level SAT attacks.

Can one scale the SAT attack to locked RTL?

Objectives

- Design an algorithm to unlock the RTL design.
- Algorithm should work on combinational and sequential circuits as well.
- Evaluates the security of a state-of-the-art RTL locking scheme.

Overall Approach

- Uses an RTL locking tool like TAO to produce locked RTL.

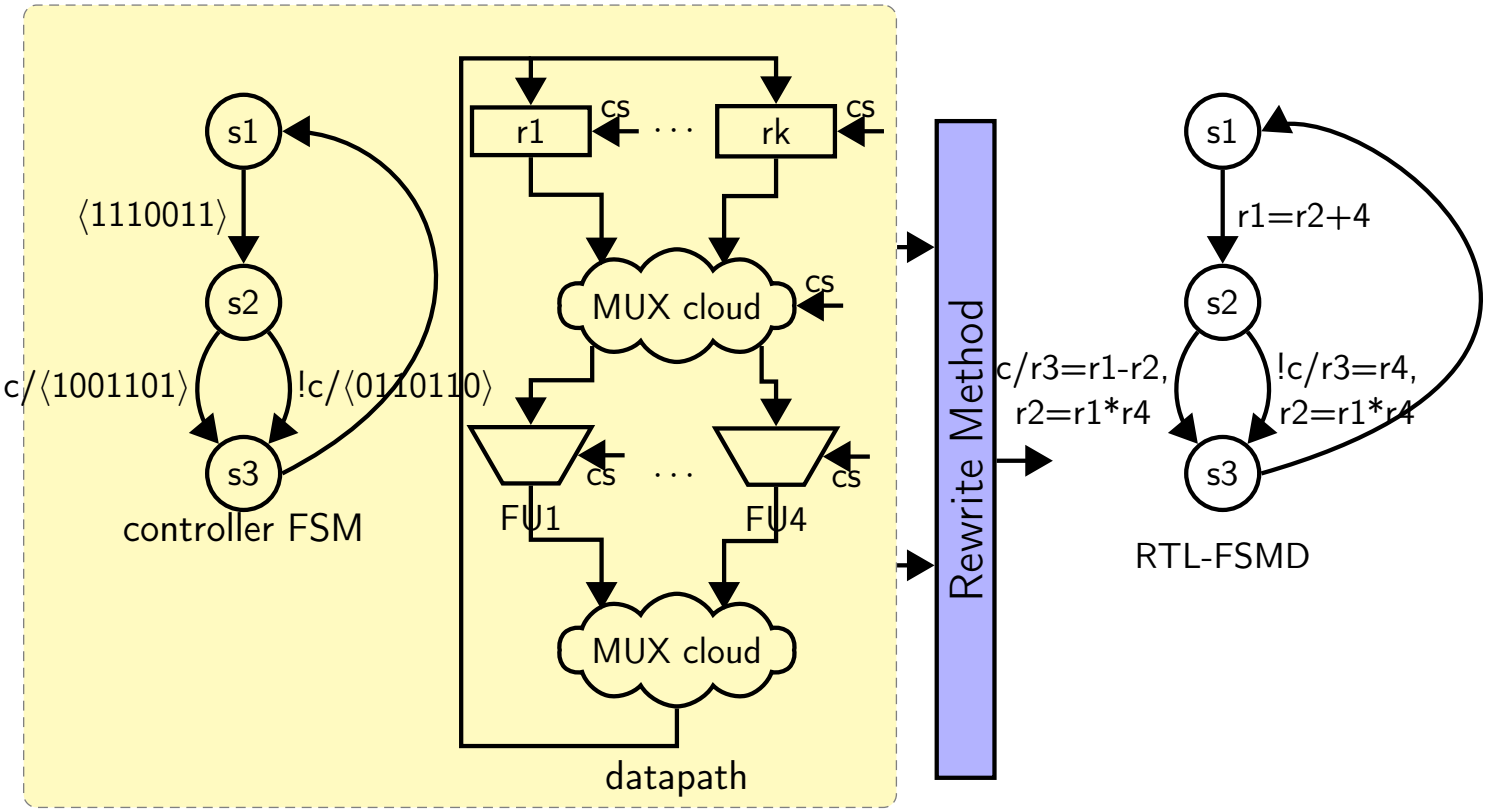
Overall Approach

- Uses an RTL locking tool like TAO to produce locked RTL.
- Models an RTL design as a RTL-FSMD by applying the rewriting approach.

Overall Approach

- Uses an RTL locking tool like TAO to produce locked RTL.
- Models an RTL design as a RTL-FSMD by applying the rewriting approach.
- Abstract out the details of the hardware into a behavioral program on which launch an SMT based attack.

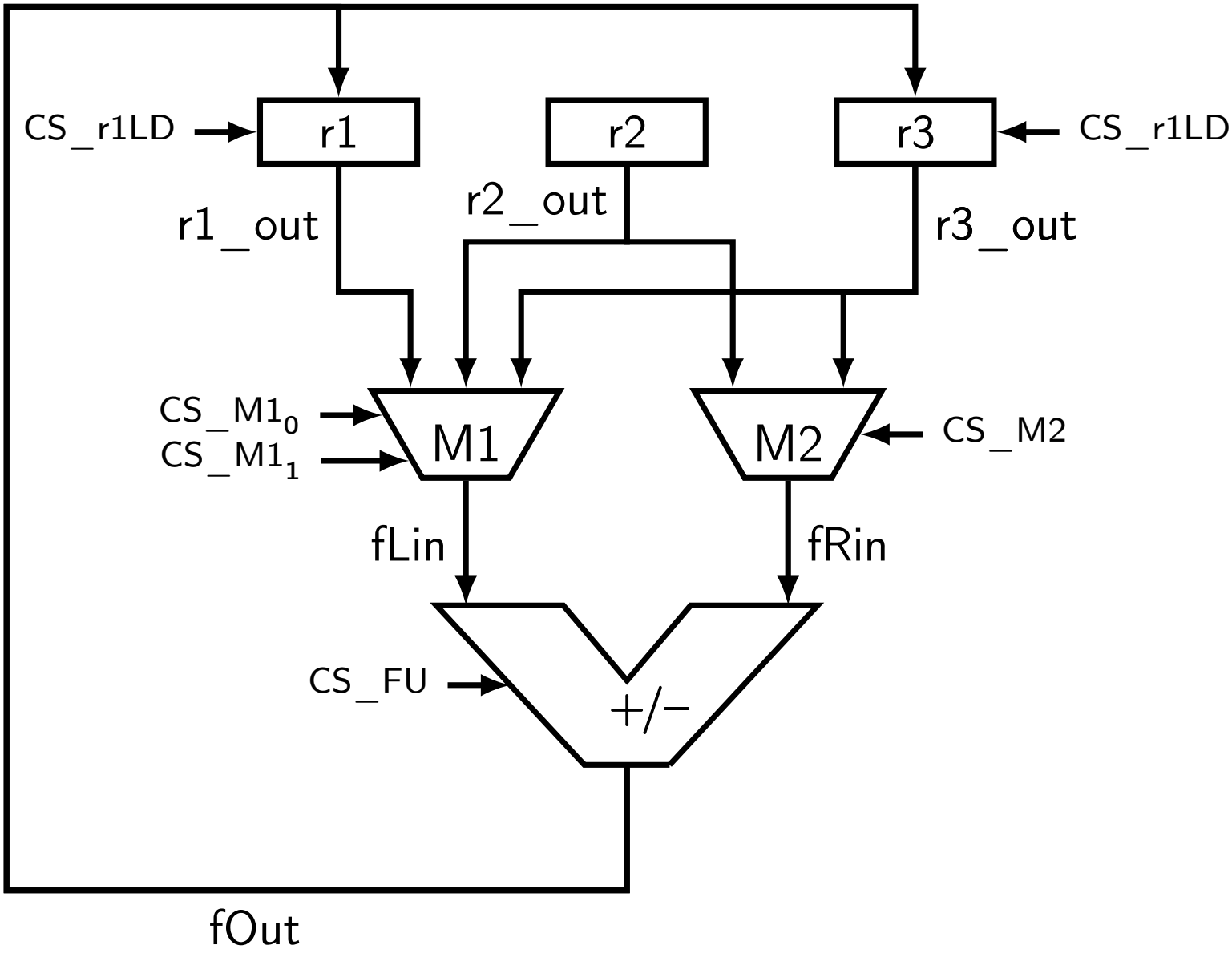
RTL-FSMD from RTL using Rewriting Method



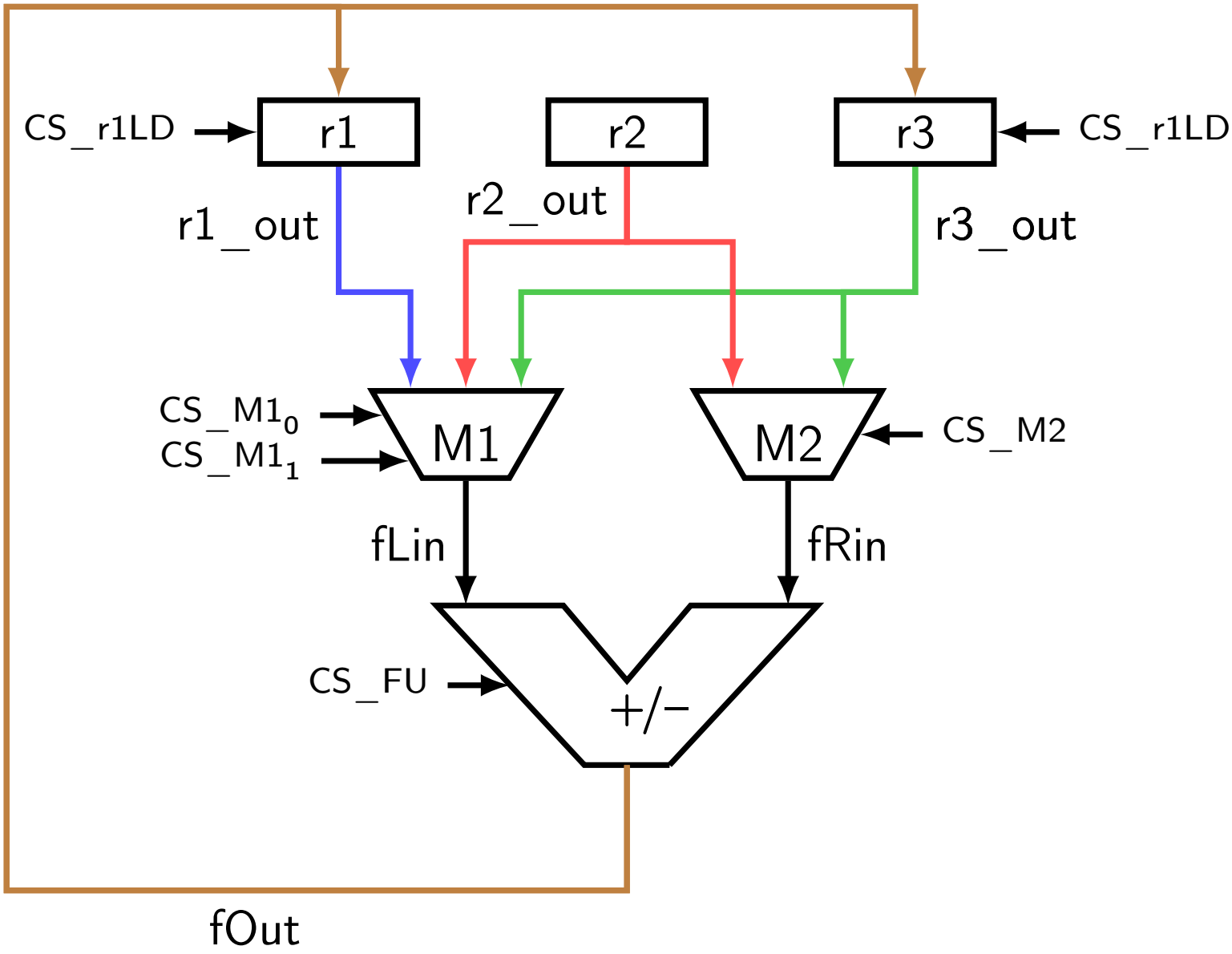
Rewrite Method

- It identifies the spatial sequence of data flow needed for an RTL operation in a reverse order.
- The micro-operations of the form $r \leftarrow r_{in}$ are found first.
- Next, r_{in} is rewritten by looking for an active micro-operation $r_{in} \leftarrow s$ or $r_{in} \leftarrow s1 < op > s2$.
- Next, s ($s1$ or $s2$ in the latter case) are rewritten provided they are not registers.
- The rewriting takes place from left to right in a breadth-first manner and terminates when all signals in the RHS expression are registers.

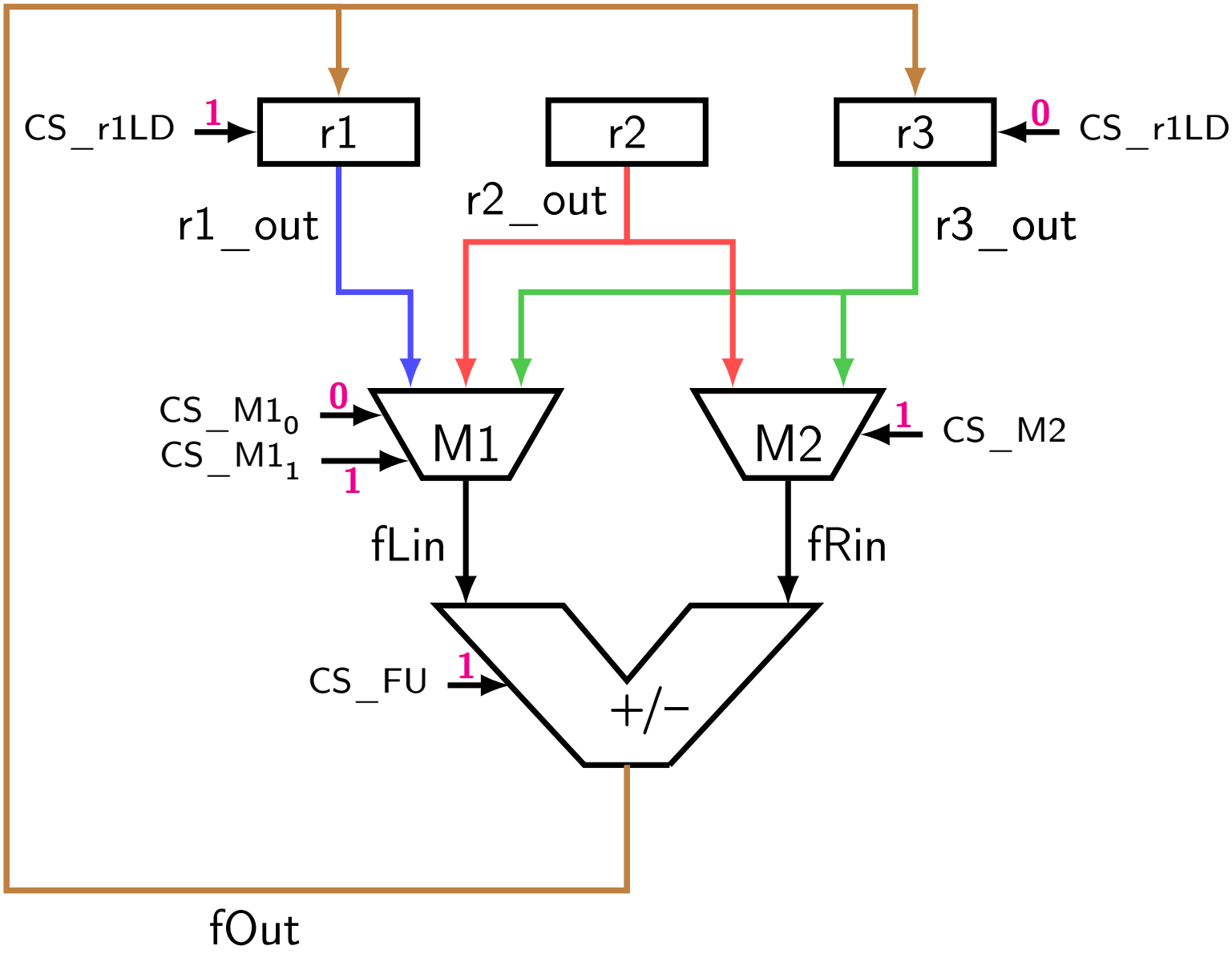
Rewrite Method



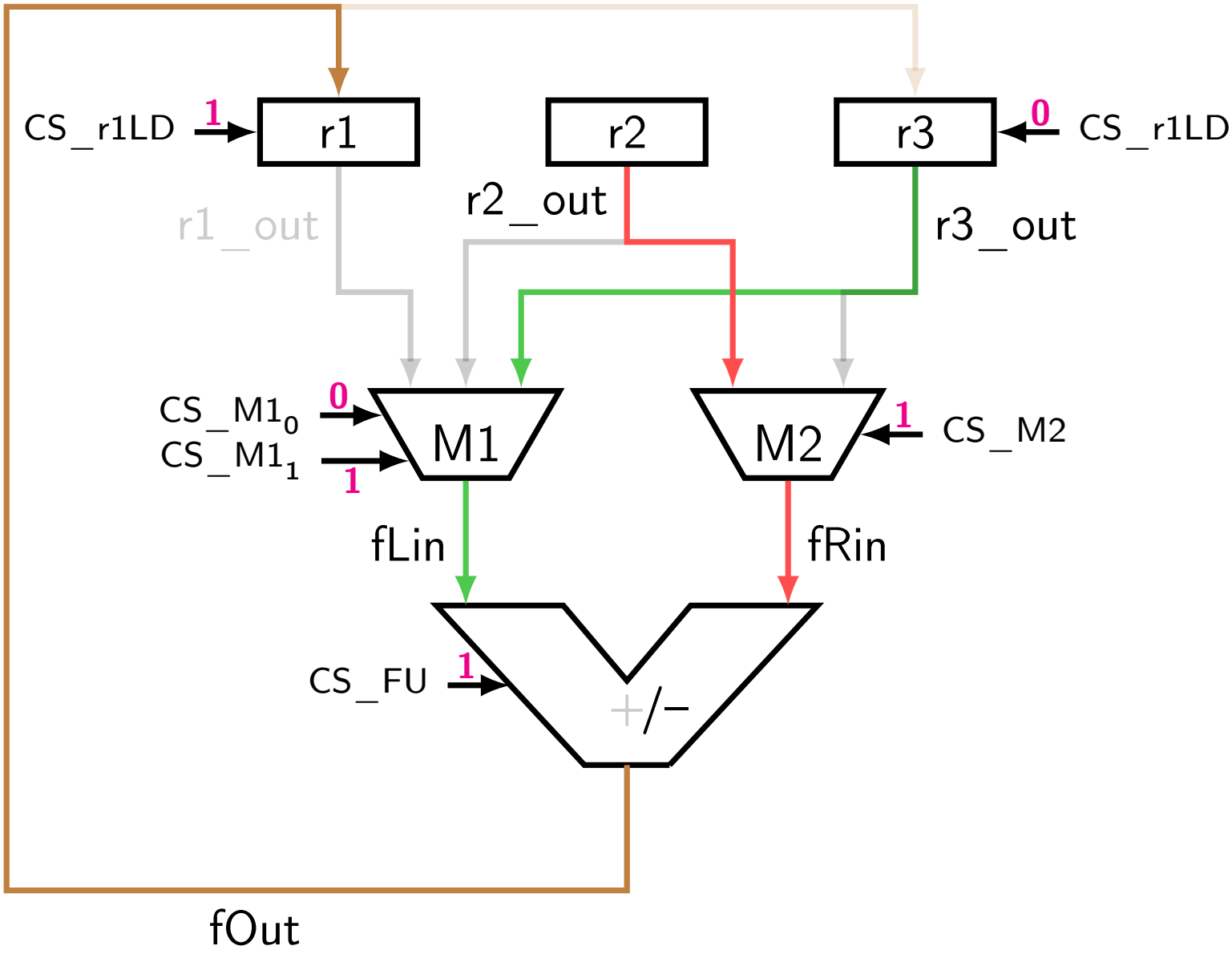
Rewrite Method



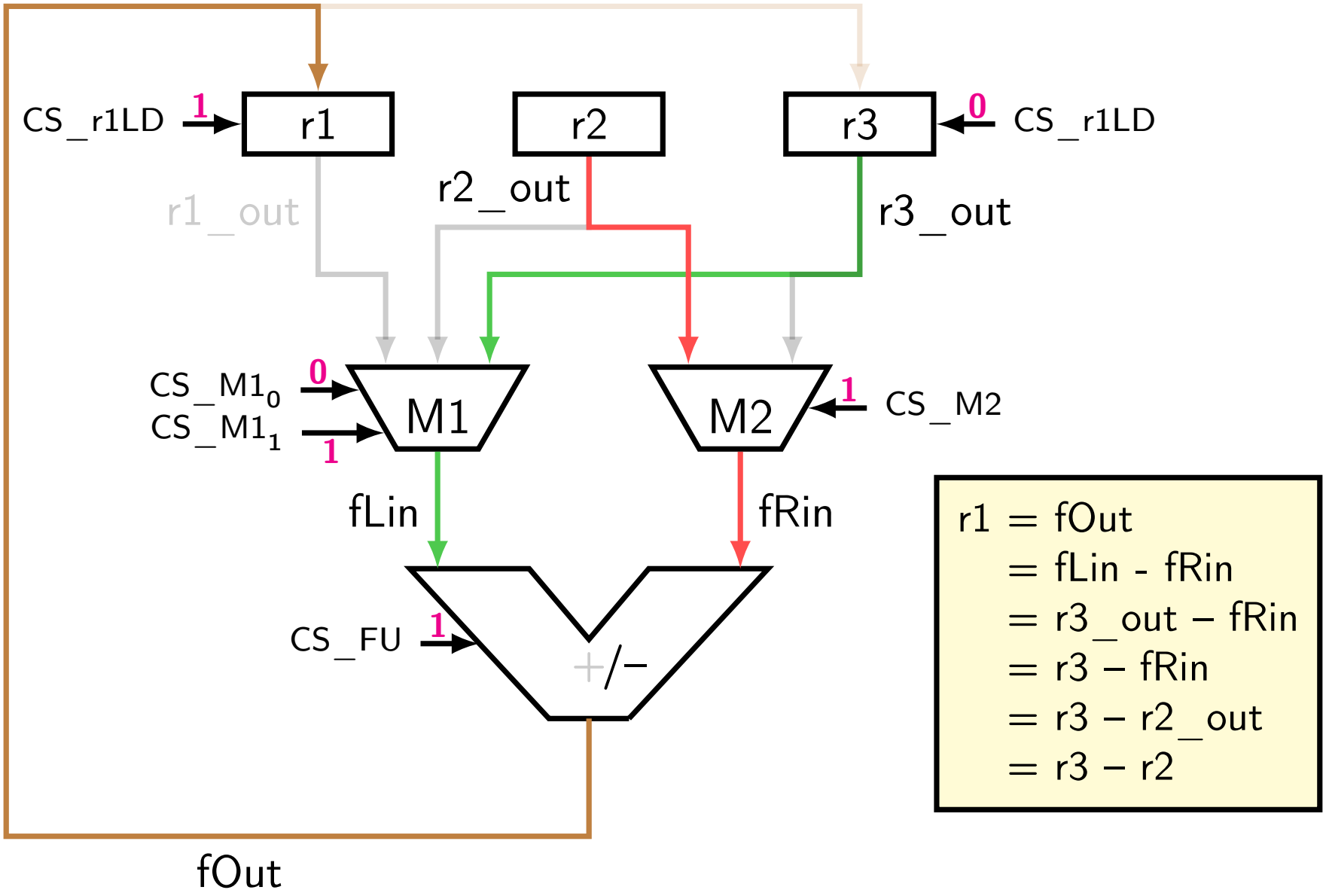
Rewrite Method



Rewrite Method



Rewrite Method



r1 = fOut
= fLin - fRin
= r3_out – fRin
= r3 – fRin
= r3 – r2_out
= r3 – r2

SMT Based Attack

- It is based on SAT attack presented by Promod et al.
- It finds distinguishing input patterns (DIPs) iteratively.
- Rule out equivalence classes of incorrect keys in each iteration.
- Stops when no DIPs are found.
- Uses Z3 SMT solver to check the satisfiablity.

Distinguishing Inputs Pattern (DIP) - Concept

$out = k1.a + k2.b + k3.c + k4.d + k5.e$ Correct Key $= \langle 1, 0, 0, 0, 1 \rangle$

DIP $= \langle 2, 2, 2, 2, 5 \rangle$ out $= 7$

K1 $= \langle 1, 1, 1, 1, 1 \rangle$ out1 $= ?$

K1 $= \langle 0, 0, 0, 0, 0 \rangle$ out2 $= ?$

$7 = 2.k1 + 2.k2 + 2.k3 + 2.k4 + 5.k5$

How many possible keys now?

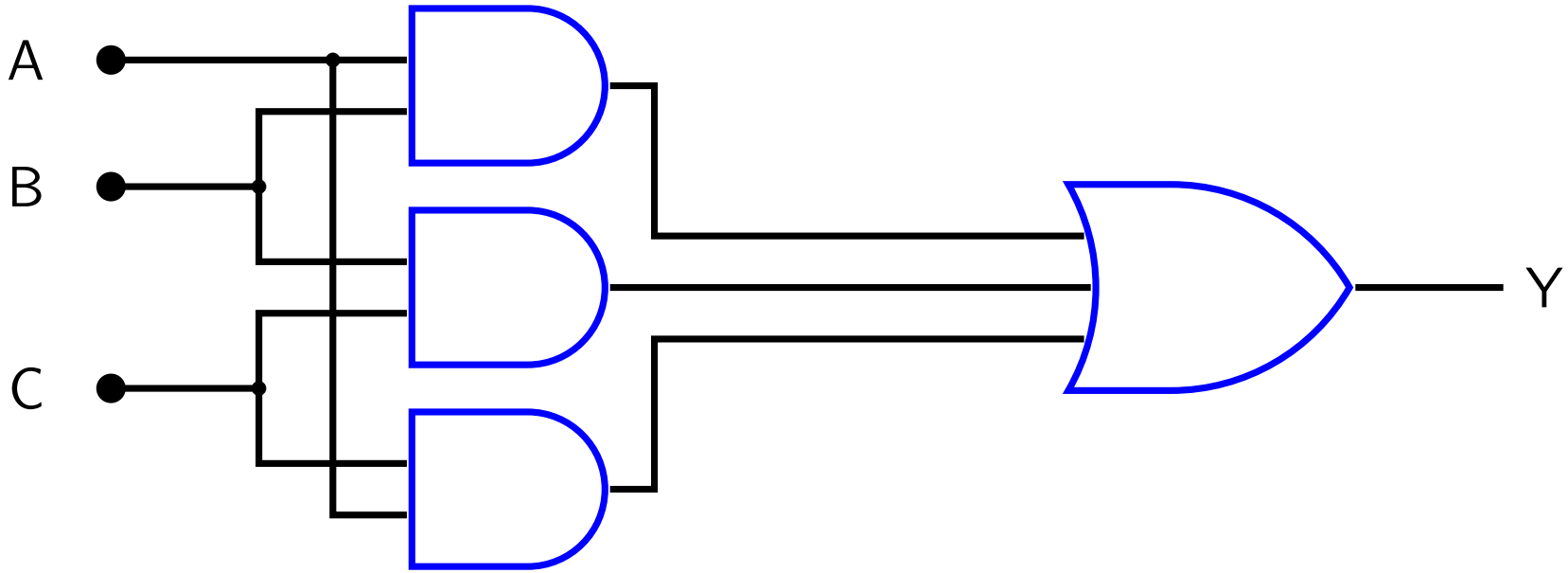
Next DIP $= \langle 2, 2, 2, 3, 5 \rangle$ out $= 7$

K1 $= \langle 0, 0, 1, 0, 1 \rangle$ out1 $= ?$

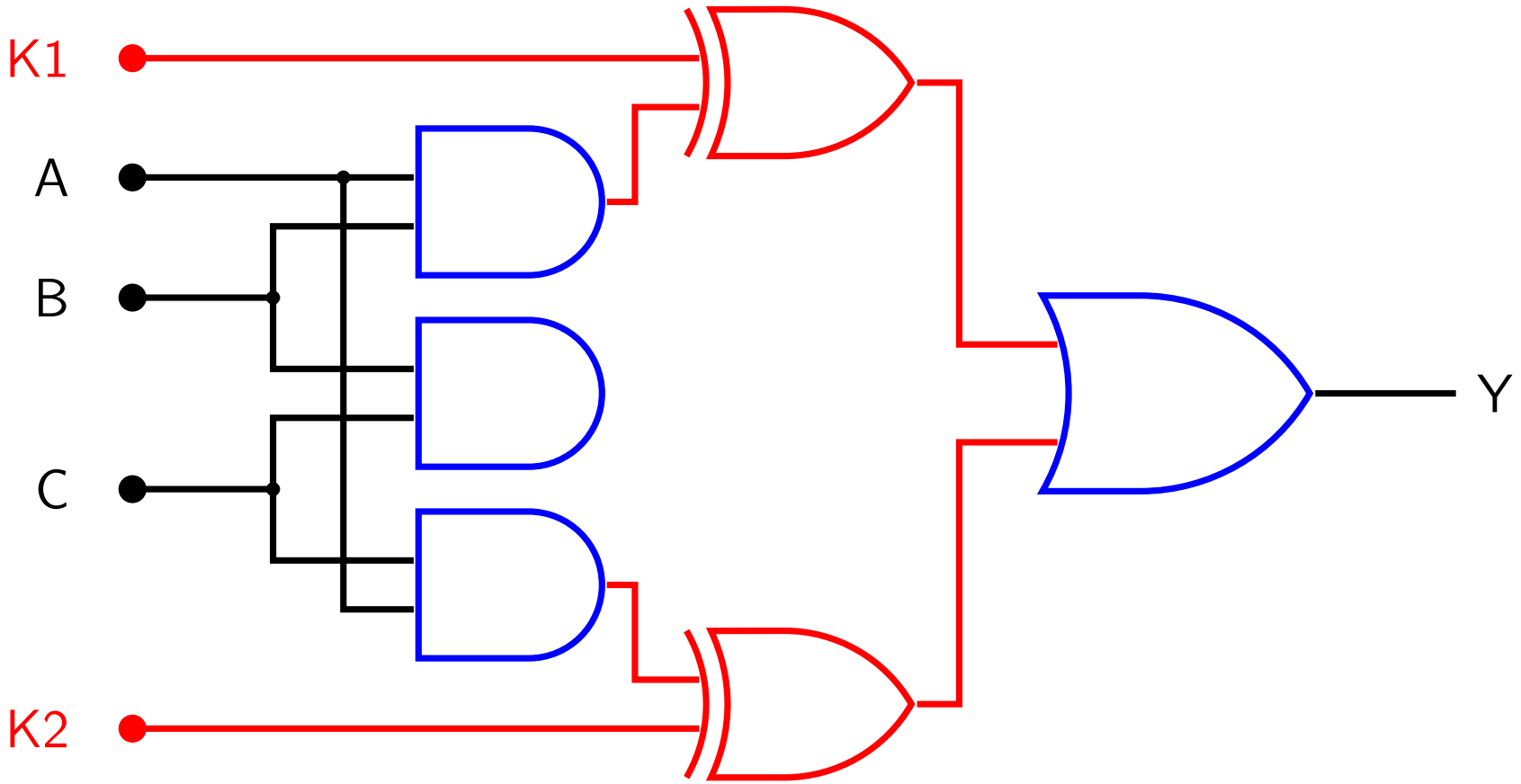
K1 $= \langle 0, 0, 0, 1, 1 \rangle$ out2 $= ?$

$7 = 2.k1 + 2.k2 + 2.k3 + 3.k4 + 5.k5$

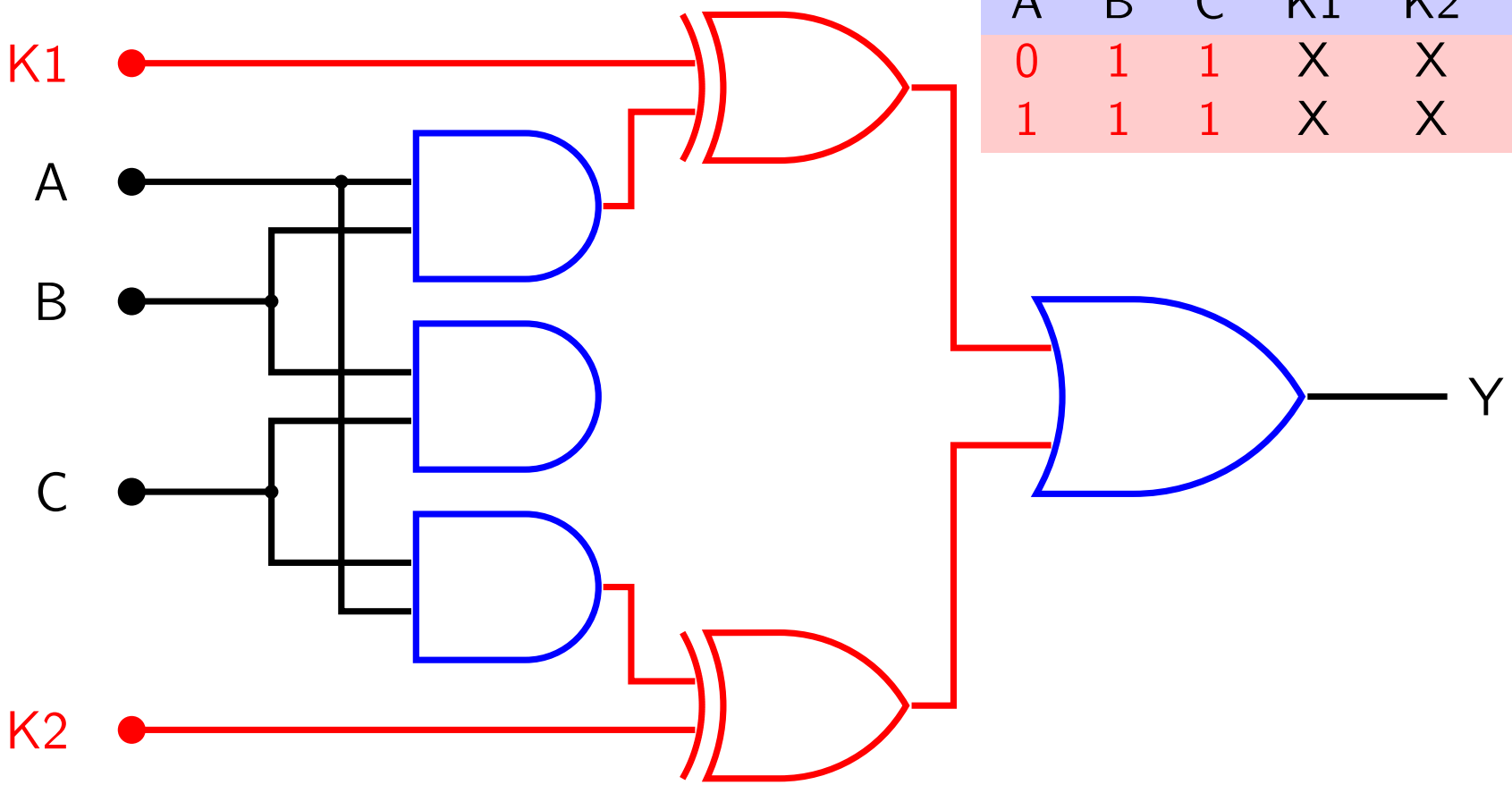
Key Idea in SAT Attack: Distinguishing Inputs



Key Idea in SAT Attack: Distinguishing Inputs



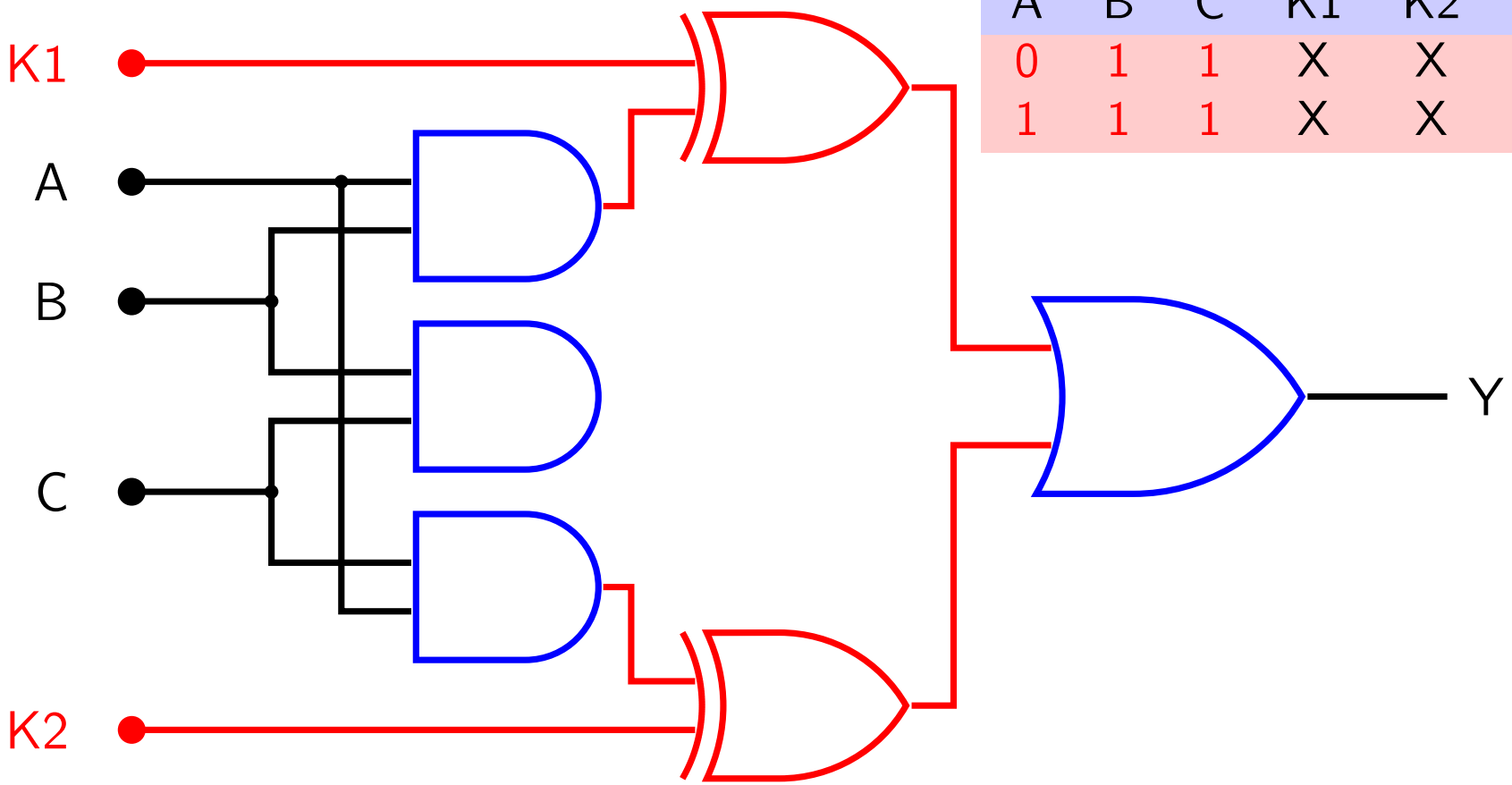
Key Idea in SAT Attack: Distinguishing Inputs



A	B	C	K1	K2	Y
0	1	1	X	X	1
1	1	1	X	X	1

The above input vectors are useless because they result in the same output value for all keys

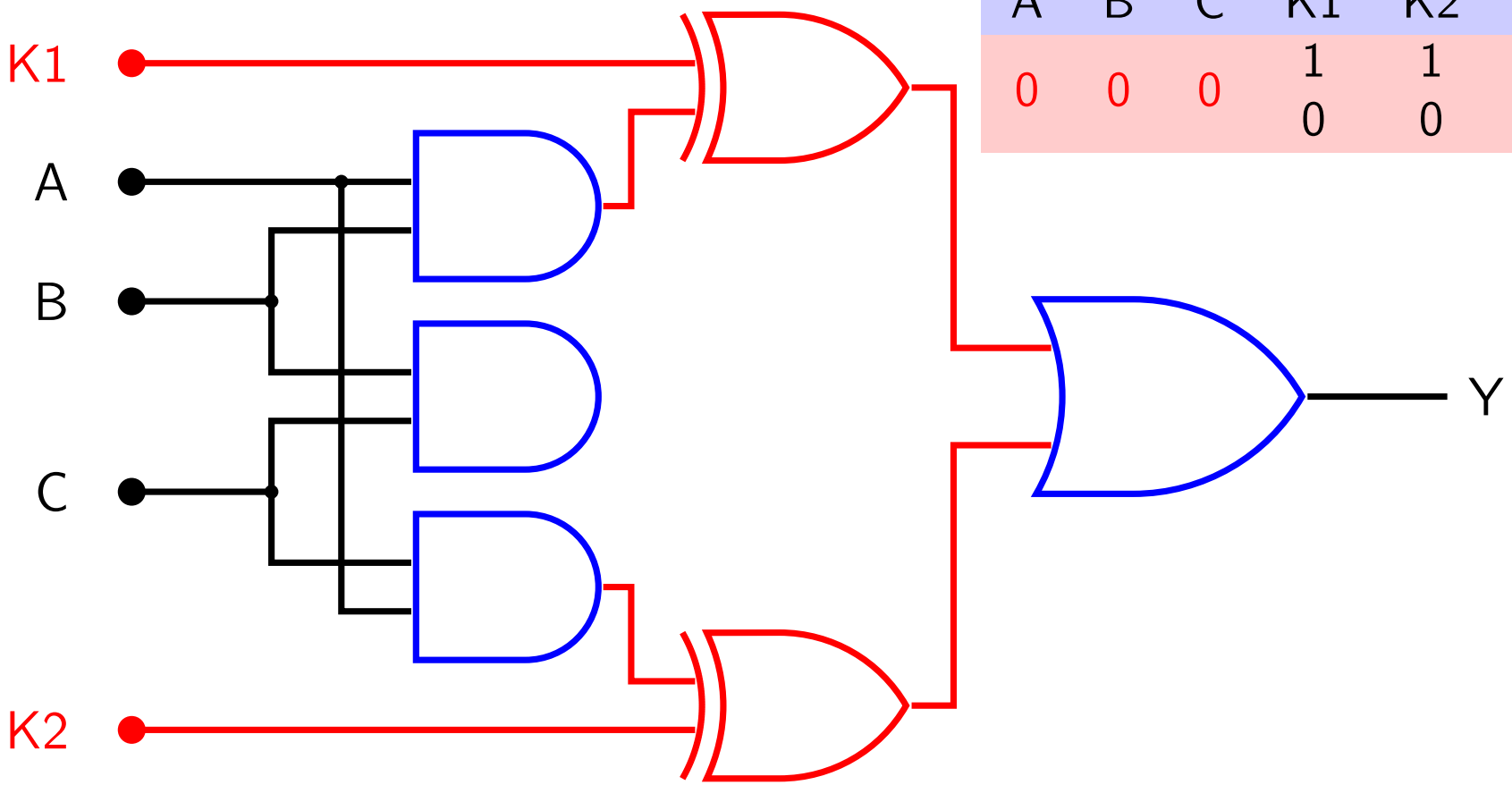
Key Idea in SAT Attack: Distinguishing Inputs



A	B	C	K1	K2	Y
0	1	1	X	X	1
1	1	1	X	X	1

Want input vectors that can distinguish between at least two keys with different behaviors

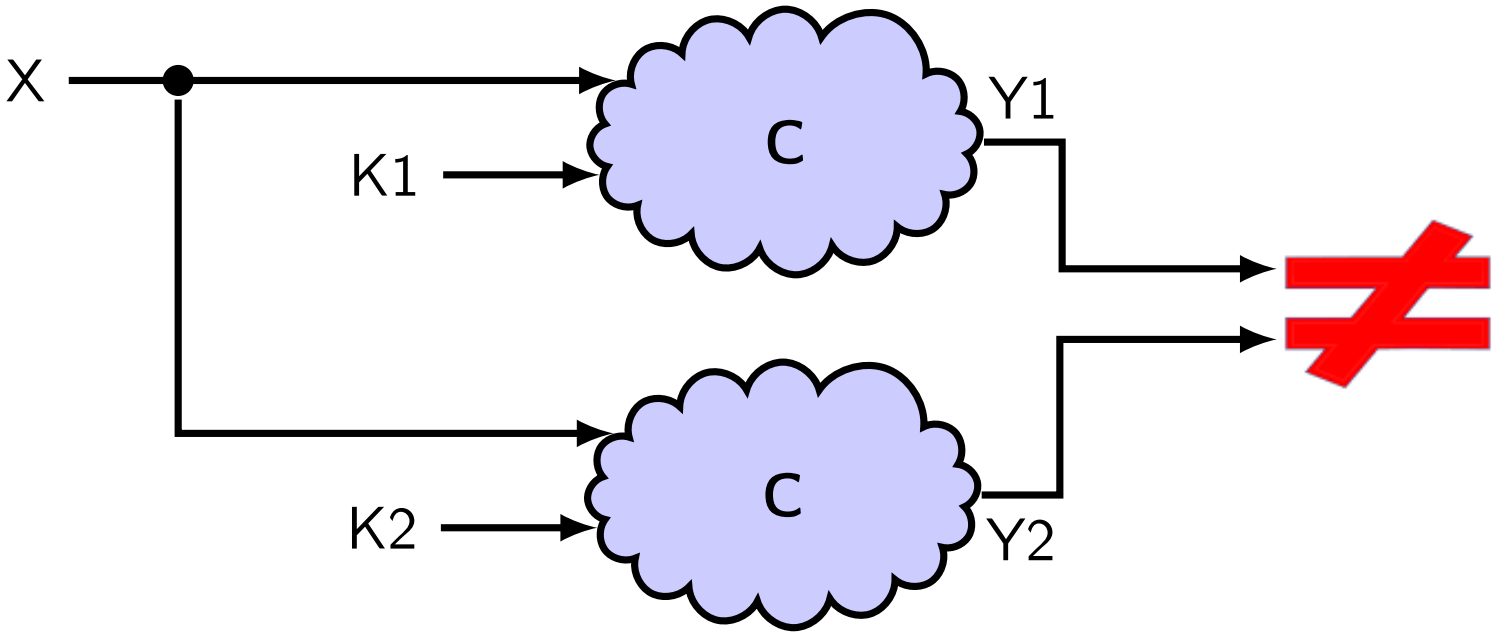
Key Idea in SAT Attack: Distinguishing Inputs



A	B	C	K1	K2	Y
0	0	0	1	1	1
			0	0	0

Distinguishing input for two key values: an input vector that produces different outputs for these keys

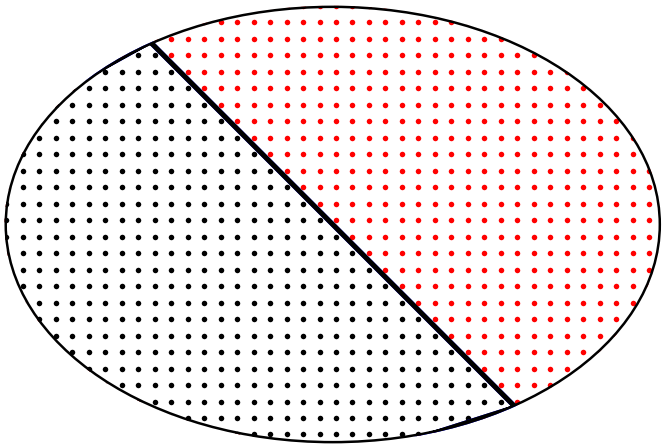
How do we compute distinguishing inputs?



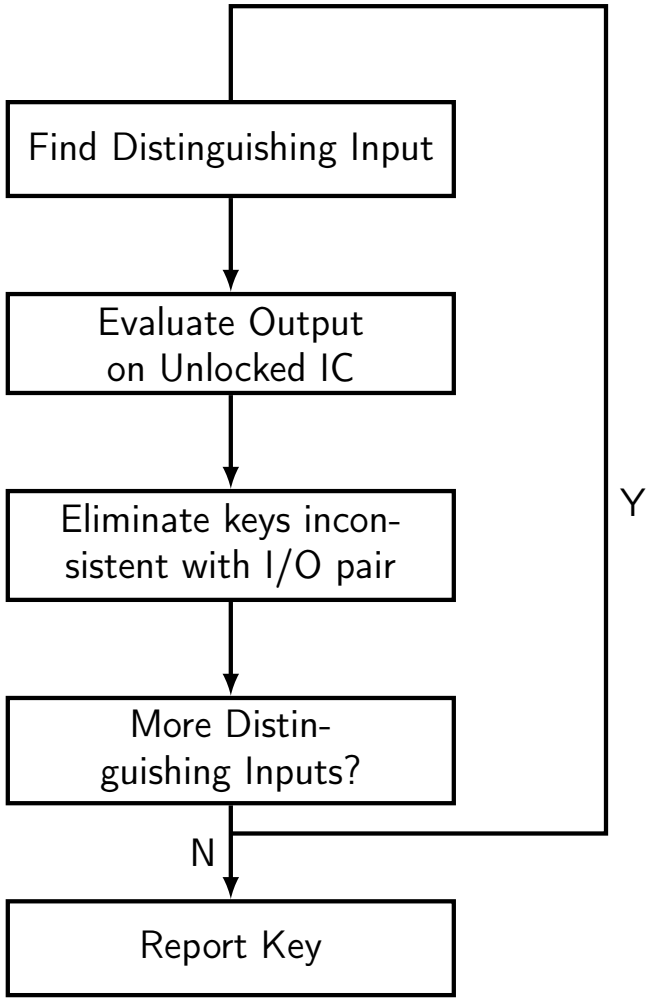
$$C(X,K1,Y1) \wedge C(X,K2,Y2) \wedge Y1 \neq Y2$$

SMT Attack

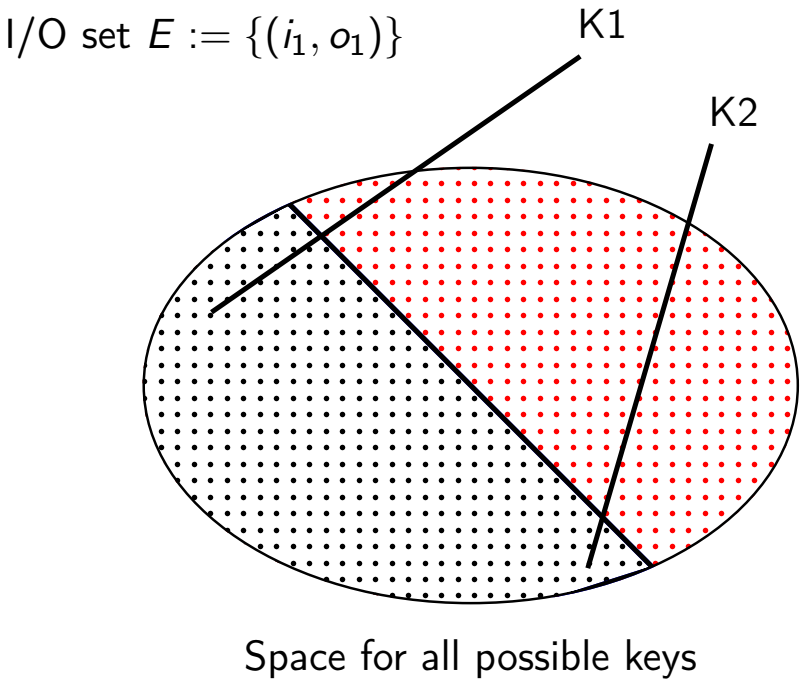
I/O set $E := \{(i_1, o_1)\}$

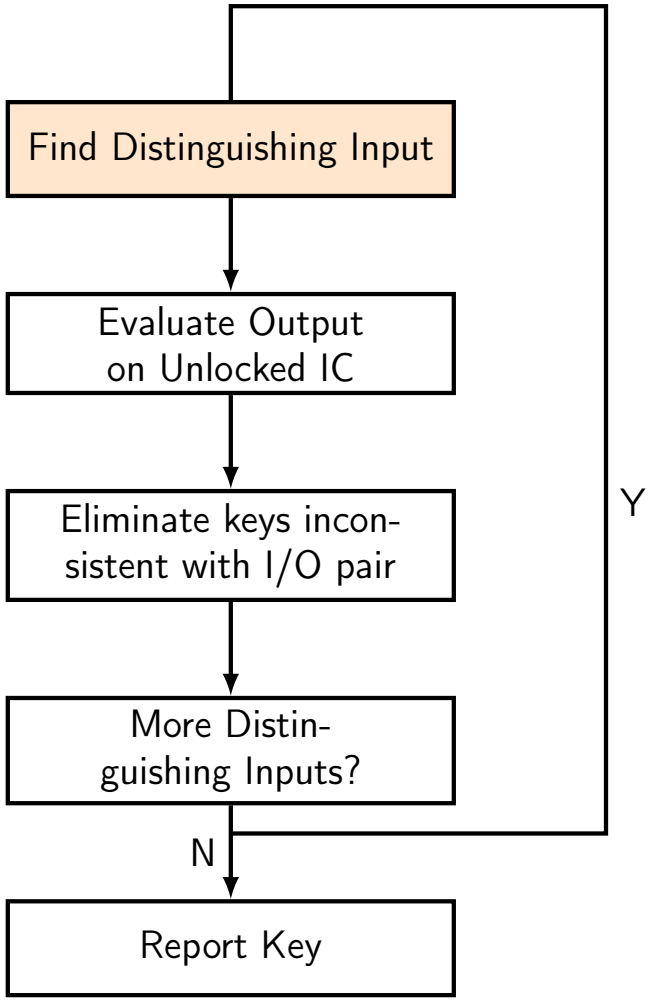


Space for all possible keys

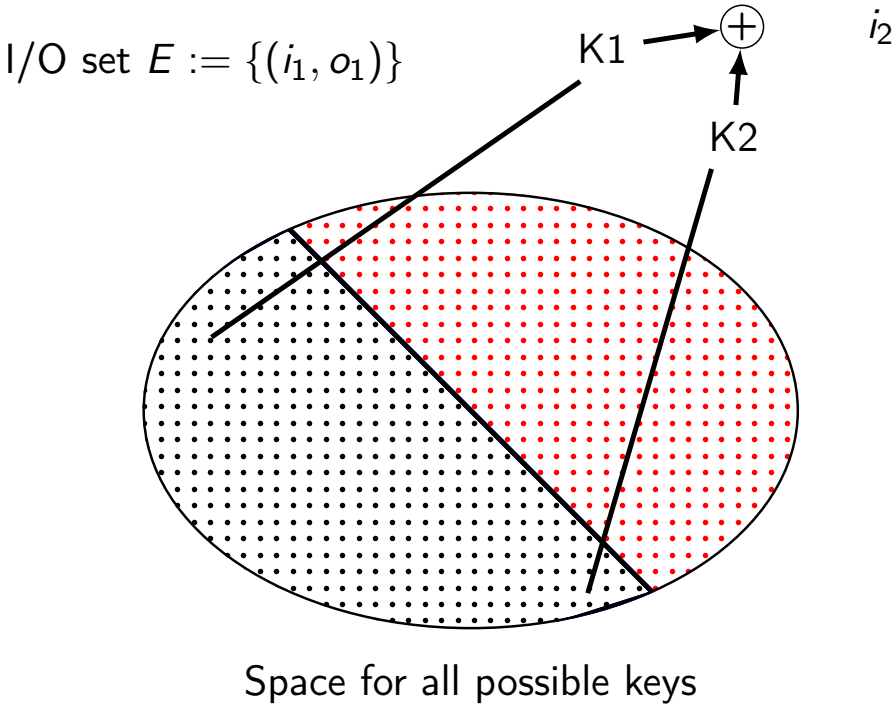


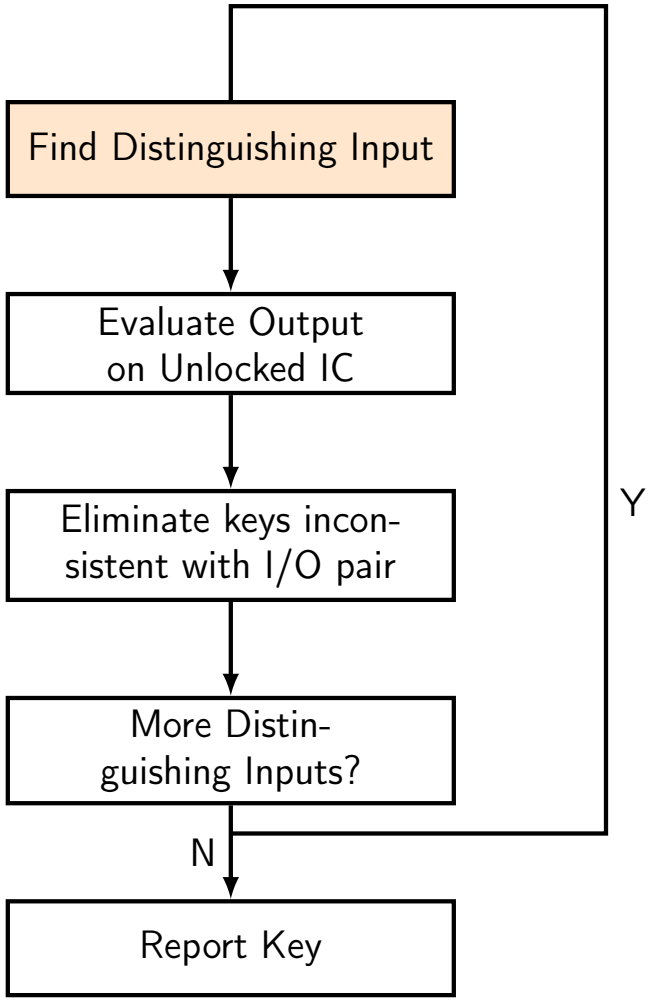
SMT Attack


I/O set $E := \{(i_1, o_1)\}$
K1
K2
Space for all possible keys

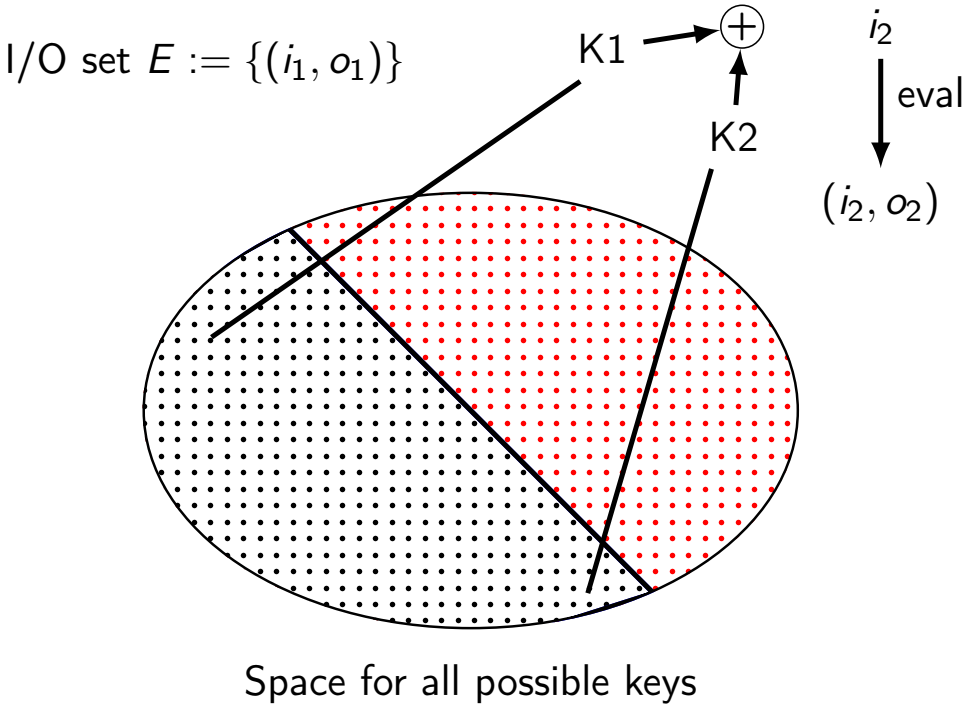

Find Distinguishing Input
Evaluate Output on Unlocked IC
Eliminate keys inconsistent with I/O pair
More Distinguishing Inputs?
Y
N
Report Key

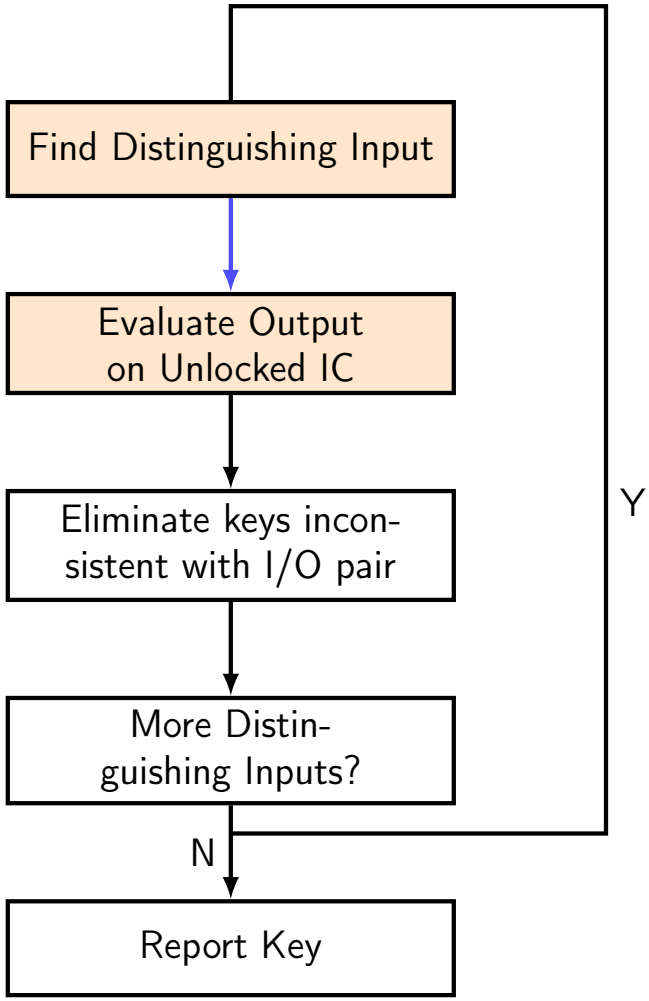
SMT Attack


I/O set $E := \{(i_1, o_1)\}$
K1
K2
i_2
Space for all possible keys


Find Distinguishing Input
Evaluate Output on Unlocked IC
Eliminate keys inconsistent with I/O pair
More Distinguishing Inputs?
Y
N
Report Key

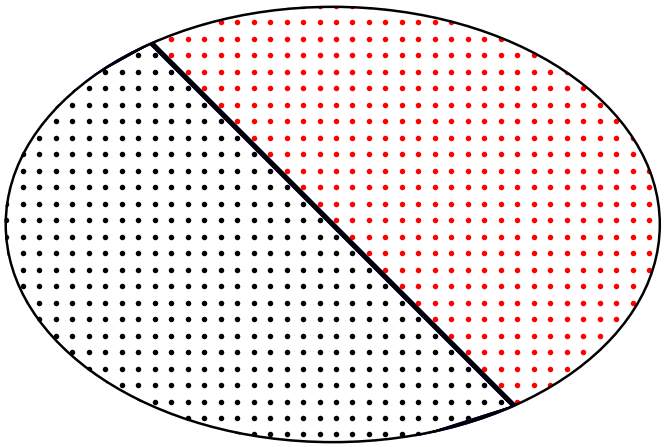
SMT Attack


I/O set $E := \{(i_1, o_1)\}$
K1
K2
i_2
eval
(i_2, o_2)
Space for all possible keys

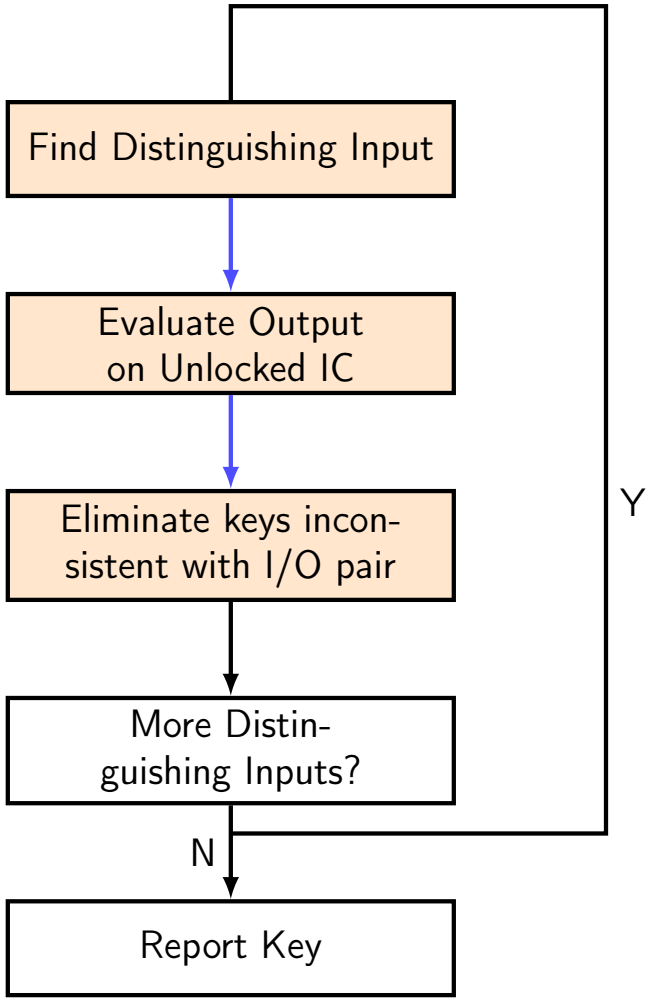

Find Distinguishing Input
Evaluate Output on Unlocked IC
Eliminate keys inconsistent with I/O pair
More Distinguishing Inputs?
Y
N
Report Key

SMT Attack

I/O set $E := E \cup \{(i_2, o_2)\}$

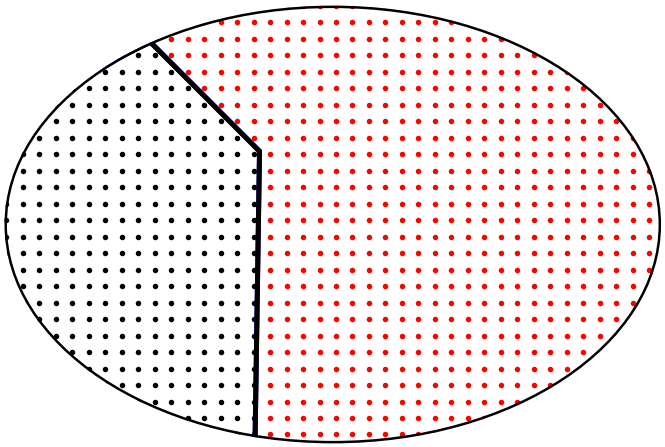


Space for all possible keys

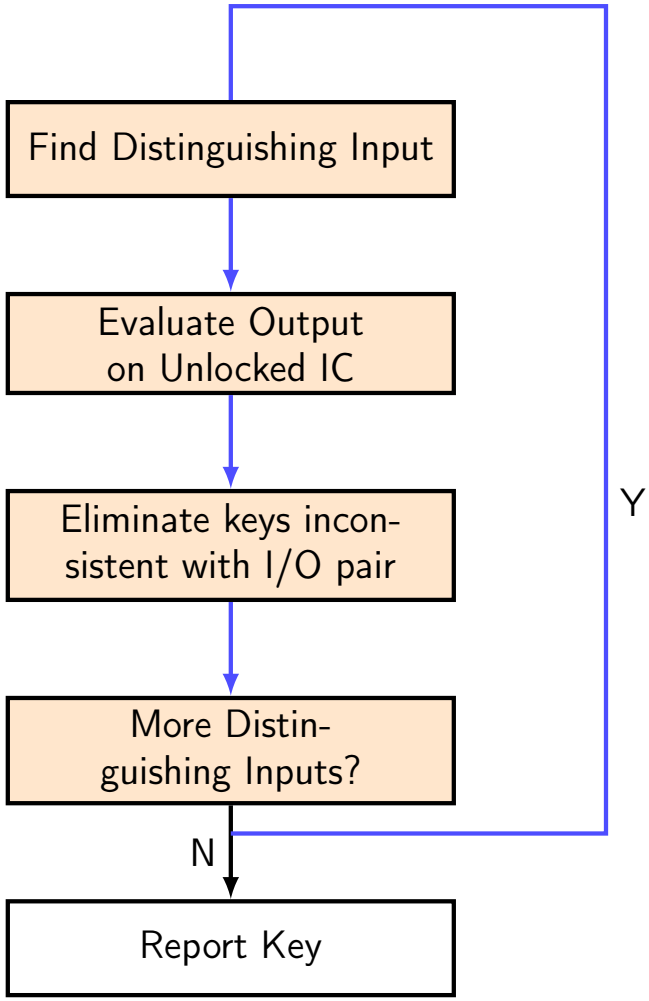


SMT Attack

I/O set $E := E \cup \{(i_j, o_j)\}$

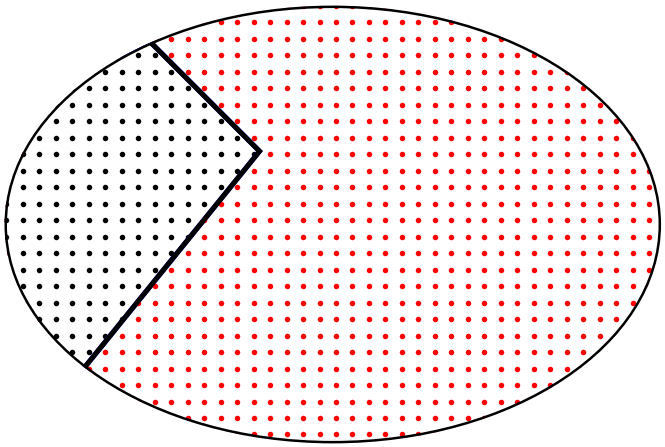


Space for all possible keys

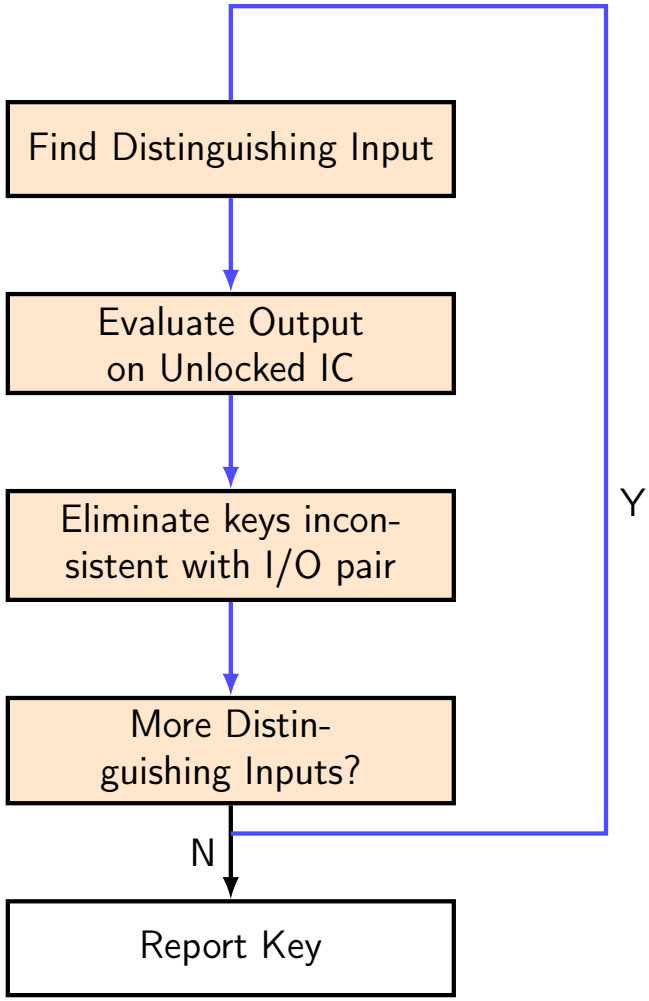


SMT Attack

I/O set $E := E \cup \{(i_k, o_k)\}$

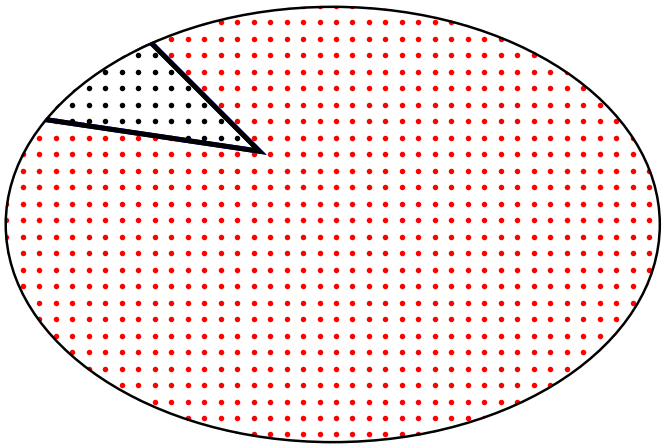


Space for all possible keys

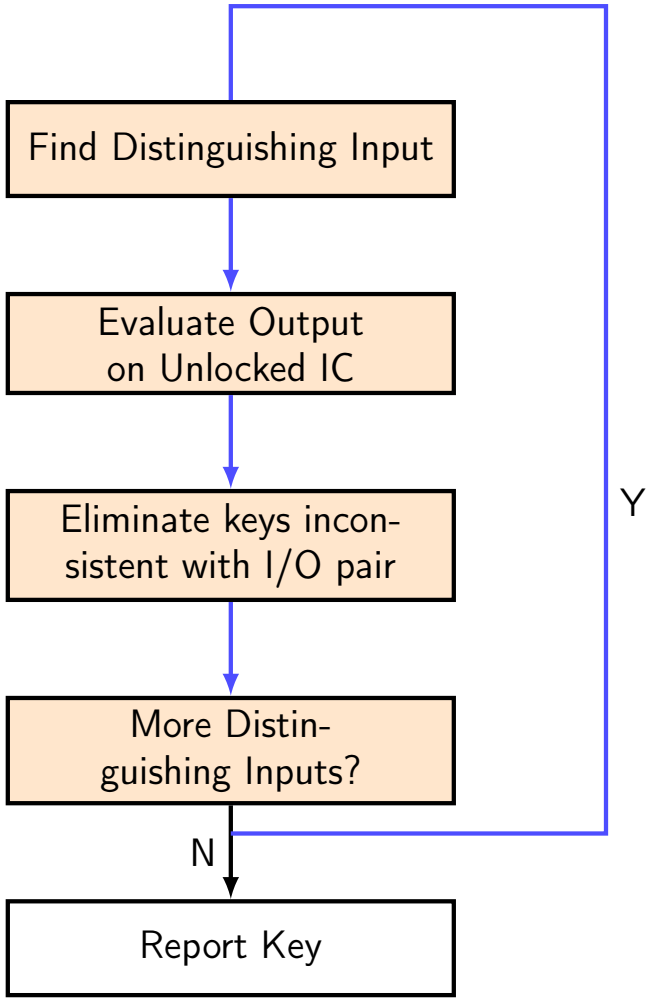


SMT Attack

I/O set $E := E \cup \{(i_I, o_I)\}$

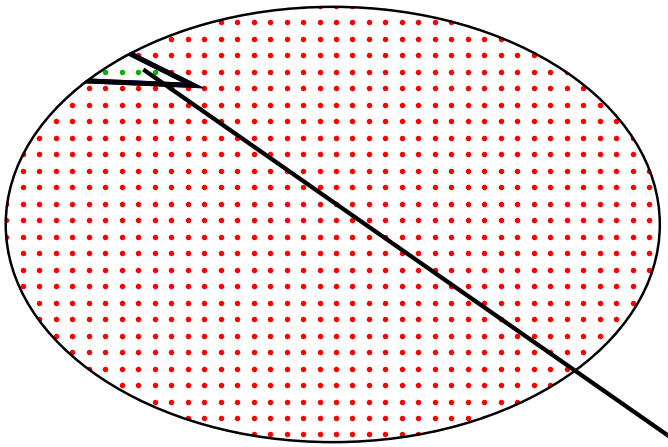


Space for all possible keys

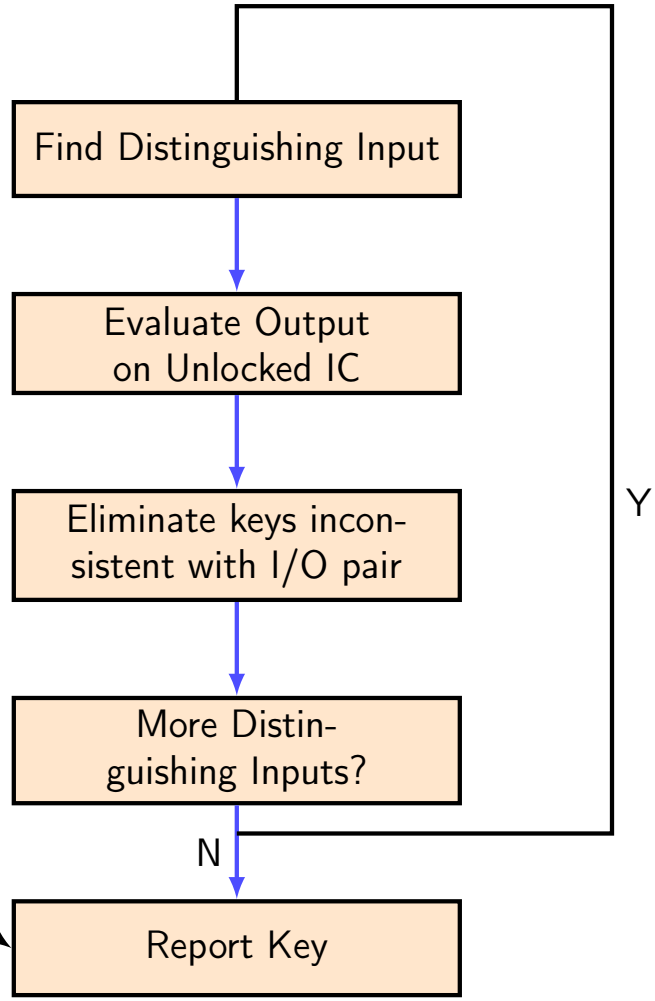


SMT Attack

I/O set $E := E \cup \{(i_n, o_n)\}$



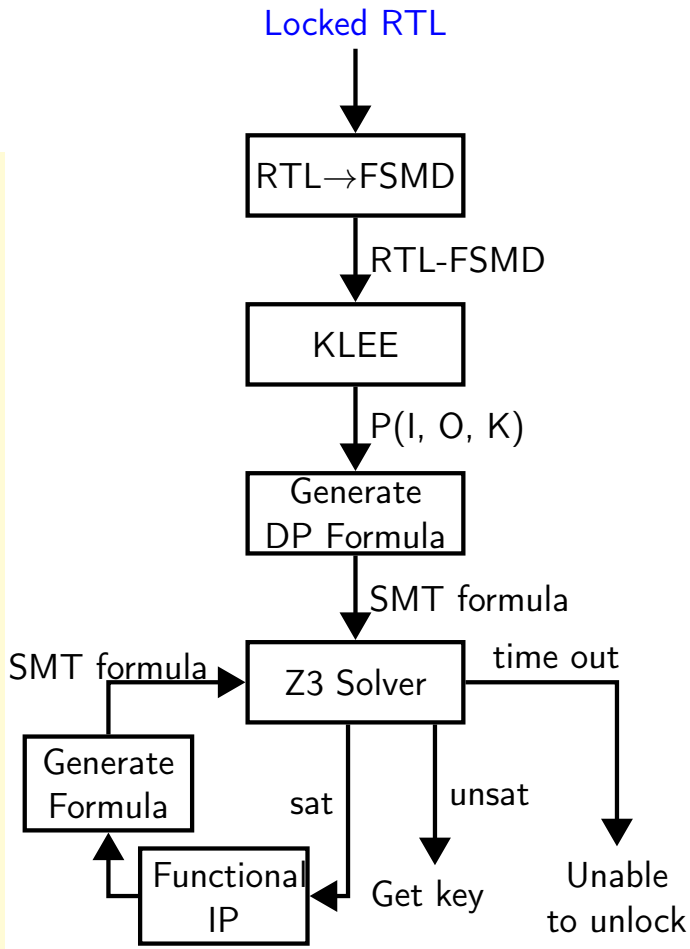
Space for all possible keys



Attack Tool-flow

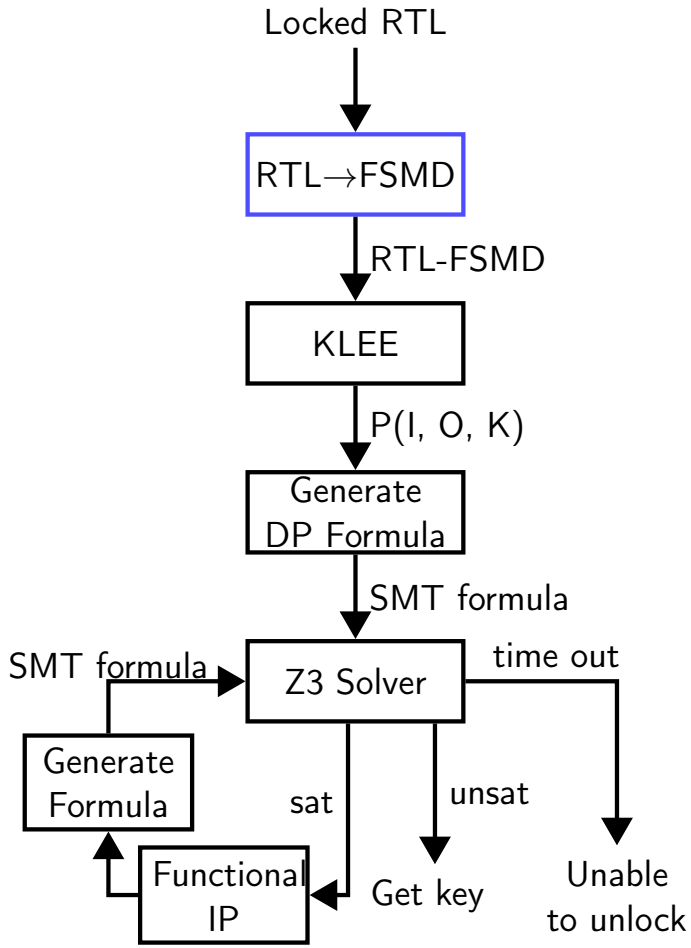
- Locked RTL generated by TAO.
- Operation, condition and constant obfuscation.

```
always @ (*) begin
 if ((1'b1 == ap_CS_fsm_state3)) begin
   ap_done = 1'b1;
  end else begin
    ap_done = 1'b0;
  end
end
always @ (*) begin
 case (ap_CS_fsm)
  ap_ST_fsm_state1 : begin
    if (((1'b1 == ap_CS_fsm_state1) &
      default : begin
        ...
      ap_NS_fsm = 'bx;
     end
    endcase
end
assign ap_CS_fsm_state3 = ap_CS_fsm[32'd2];
assign c_assign_fu_69_p1 = b;
assign c_assign_fu_69_p2 =
(($signed(a) > $signed(c_assign_fu_69_p1))
? 1'b1 : 1'b0);
```



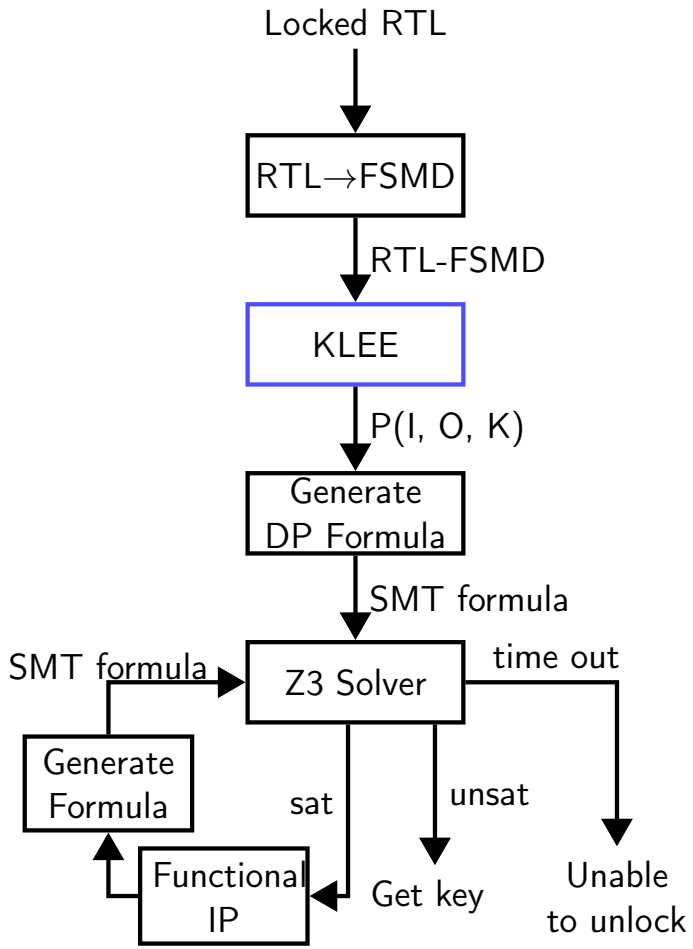
Attack Tool-flow

- Parses the locked RTL generated by TAO using Pyverilog.
- Uses a rewriting method yielding an RTL-FSMD.
- The RTL-FSMD $P(I, O, K) \in \mathbb{Z}^{M+N+K}$ has M primary inputs, N primary outputs and K unknown keys.
- It represents the input/output relation of the locked RTL design based on the key.



Attack Tool-flow

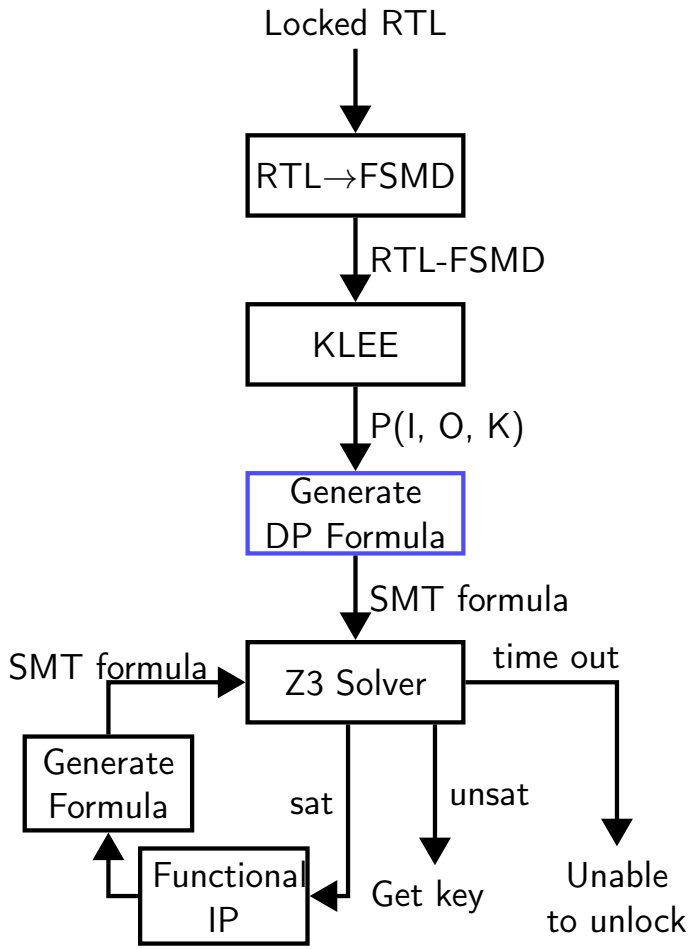
- Transforms the RTL-FSMD to feed into the KLEE tool to get the symbolic representation of the outputs.
- The RTL-FSMD consists of a finite set of traces $\{\tau_1, \tau_2, \ldots, \tau_k\}$.
- The outputs O in the RTL-FSMD can be represented as $P(I, O, K): \; O = (ite\ C_{\tau_1}\ D_{\tau_1}\ (ite\ C_{\tau_2} D_{\tau_2}\ (ite\ \ldots (ite\ C_{\tau_{k-1}}\ D_{\tau_{k-1}} D_{\tau_k}))\ldots))$ where $(ite\ C\ D_1\ D_2)$ (aka if-then-else) indicates if the condition C is `True` return the value D_1 else D_2.



Attack Tool-flow

- Creates the SAT formulation for DIP.

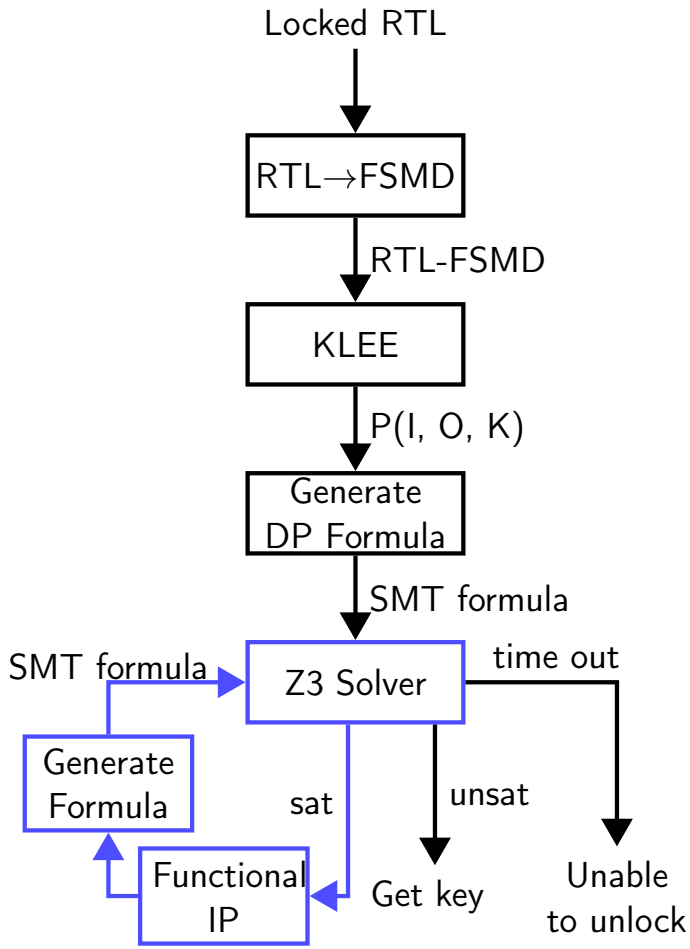
```
;c = a > b
;if(c xor c1)
; out = a + k1
;else
; out = b * k2
(define-fun G ((a Int) (b Int)) Bool (> a b))
(define-fun A ((a Int) (b Int) (x1 Int) (x2 Int)
               (x3 Bool) (c Bool)) Int (ite
               (xor c x3) (+ a x1) (* b x2)))
(assert (= out1 (A a b k11 k21 c1 (G a b))))
(assert (= out2 (A a b k12 k22 c2 (G a b))))
(assert (not (= out1 out2)))
(check-sat)
(get-model)
```



Attack Tool-flow

```
;c = a > b
;if(c xor c1)
; out = a + k1
;else
; out = b * k2
(define-fun G ((a Int) (b Int)) Bool (> a b))
(define-fun A ((a Int) (b Int) (x1 Int) (x2 Int)
               (x3 Bool) (c Bool)) Int (ite
               (xor c x3) (+ a x1) (* b x2)))
(assert (= out1 (A a b k11 k21 c1 (G a b))))
(assert (= out2 (A a b k12 k22 c2 (G a b))))
(assert (not (= out1 out2)))
(check-sat)
(get-model)

;Iteration 2:  a = 0, b = 0 → out = 0
;added assertions
(assert (= 0 (A 0 0 k11 k21 c1 (G 0 0))))
(assert (= 0 (A 0 0 k12 k22 c2 (G 0 0))))
```

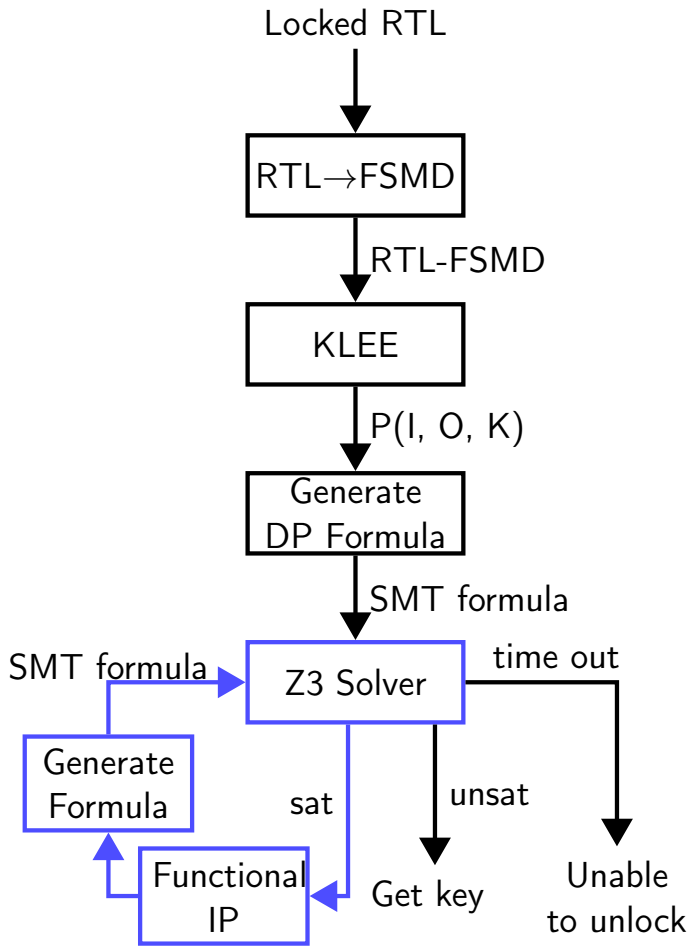


Attack Tool-flow

```
;c = a > b
;if(c xor c1)
; out = a + k1
;else
; out = b * k2
(define-fun G ((a Int) (b Int)) Bool (> a b))
(define-fun A ((a Int) (b Int) (x1 Int) (x2 Int)
               (x3 Bool) (c Bool)) Int (ite
               (xor c x3) (+ a x1) (* b x2)))
(assert (= out1 (A a b k11 k21 c1 (G a b))))
(assert (= out2 (A a b k12 k22 c2 (G a b))))
(assert (not (= out1 out2)))
(check-sat)
(get-model)

;Iteration 2:  a = 0, b = 0 → out = 0
;added assertions
(assert (= 0 (A 0 0 k11 k21 c1 (G 0 0))))
(assert (= 0 (A 0 0 k12 k22 c2 (G 0 0))))

Iteration 3:  a = -5, b = -1 → out = -3
;added assertions
(assert (= -3 (A -5 -1 k11 k21 c1 (G -5 -1))))
(assert (= -3 (A -5 -1 k12 k22 c2 (G -5 -1))))
```



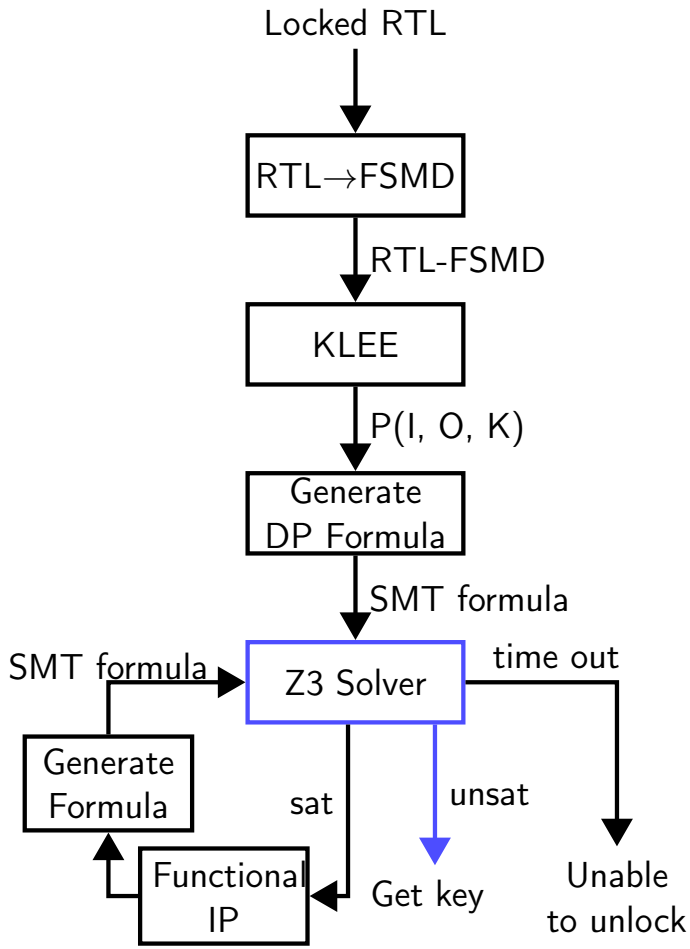
Attack Tool-flow

```
;c = a > b
;if(c xor c1)
; out = a + k1
;else
; out = b * k2
(define-fun G ((a Int) (b Int)) Bool (> a b))
(define-fun A ((a Int) (b Int) (x1 Int) (x2 Int)
               (x3 Bool) (c Bool)) Int (ite
               (xor c x3) (+ a x1) (* b x2)))
(assert (= out1 (A a b k11 k21 c1 (G a b))))
(assert (= out2 (A a b k12 k22 c2 (G a b))))
(assert (not (= out1 out2)))
(check-sat)
(get-model)

;Iteration 2:  a = 0, b = 0 → out = 0
;added assertions
(assert (= 0 (A 0 0 k11 k21 c1 (G 0 0))))
(assert (= 0 (A 0 0 k12 k22 c2 (G 0 0))))

Iteration 3:  a = -5, b = -1 → out = -3
;added assertions
(assert (= -3 (A -5 -1 k11 k21 c1 (G -5 -1))))
(assert (= -3 (A -5 -1 k12 k22 c2 (G -5 -1))))

Iteration 4:  a = -3, b = -4 → out = 2
;added assertions
(assert (= 2 (A -3 -4 k11 k21 c1 (G -3 -4))))
(assert (= 2 (A -3 -4 k12 k22 c2 (G -3 -4))))
```



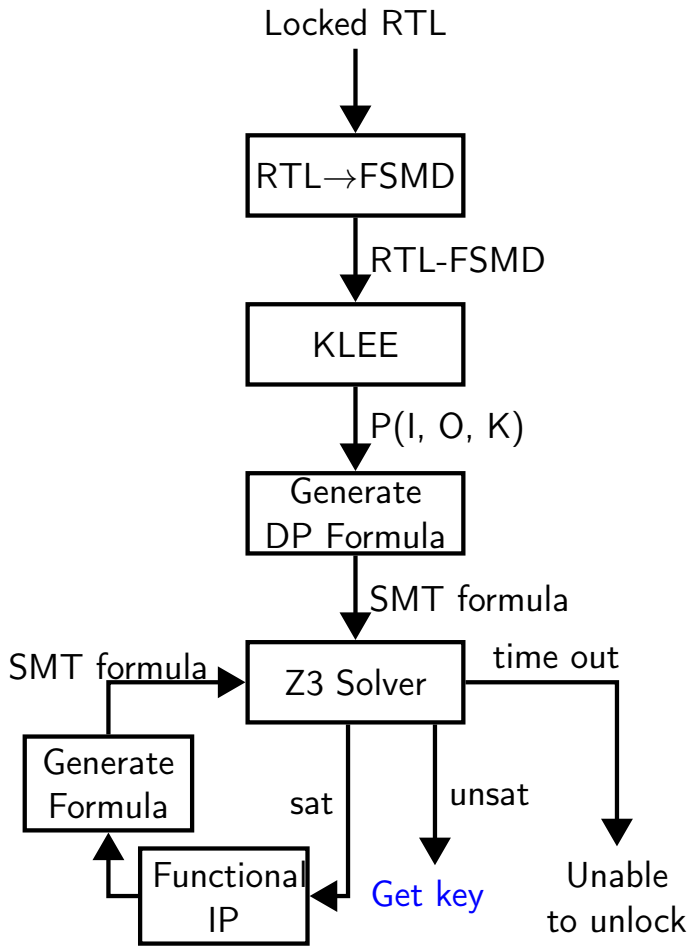
Attack Tool-flow

```
;c = a > b
;if(c xor c1)
; out = a + k1
;else
; out = b * k2
(define-fun G ((a Int) (b Int)) Bool (> a b))
(define-fun A ((a Int) (b Int) (x1 Int) (x2 Int)
               (x3 Bool) (c Bool)) Int (ite
               (xor c x3) (+ a x1) (* b x2)))
(assert (= out1 (A a b k11 k21 c1 (G a b))))
(assert (= out2 (A a b k12 k22 c2 (G a b))))
(assert (not (= out1 out2)))
(check-sat)
(get-model)

;Iteration 2:  a = 0, b = 0 → out = 0
;added assertions
(assert (= 0 (A 0 0 k11 k21 c1 (G 0 0))))
(assert (= 0 (A 0 0 k12 k22 c2 (G 0 0))))

Iteration 3:  a = -5, b = -1 → out = -3
;added assertions
(assert (= -3 (A -5 -1 k11 k21 c1 (G -5 -1))))
(assert (= -3 (A -5 -1 k12 k22 c2 (G -5 -1))))

Iteration 4:  a = -3, b = -4 → out = 2
;added assertions
(assert (= 2 (A -3 -4 k11 k21 c1 (G -3 -4))))
(assert (= 2 (A -3 -4 k12 k22 c2 (G -3 -4))))
```



k1 = 5, k2 = 3, c1 = False by reversing SAT (i.e., (assert (= out1 out2)))

Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

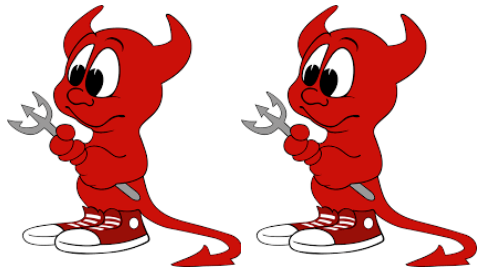
Blue Team

Christian Siddharth Ramesh



RedTeam

Chandan Ramanuj



Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

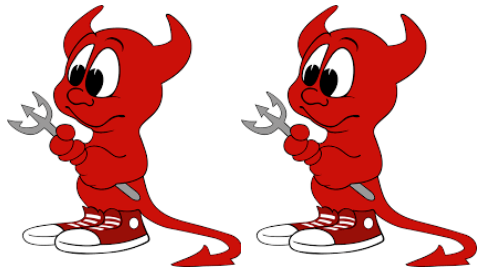
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.

RedTeam

Chandan Ramanuj



Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

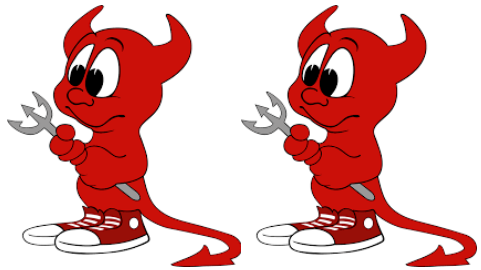
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.
- TAO applies operation, control and constant locking.

RedTeam

Chandan Ramanuj



Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

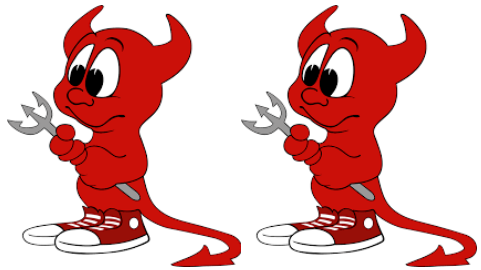
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.
- TAO applies operation, control and constant locking.
- Large design with 14K combinational cells and 3K sequential cells.

RedTeam

Chandan Ramanuj



Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

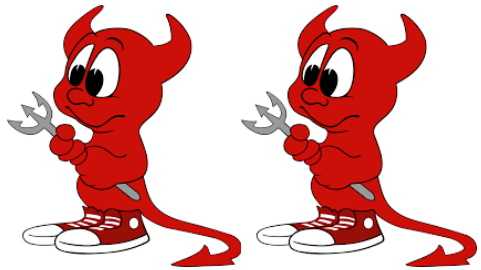
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.
- TAO applies operation, control and constant locking.
- Large design with 14K combinational cells and 3K sequential cells.

RedTeam

Chandan Ramanuj



- Applied SMT based attack.

Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

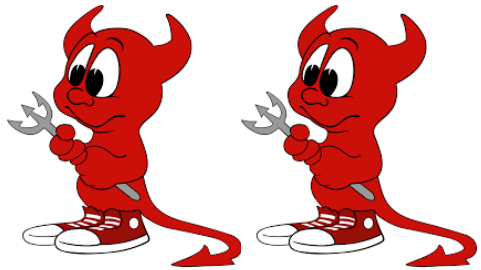
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.
- TAO applies operation, control and constant locking.
- Large design with 14K combinational cells and 3K sequential cells.

RedTeam

Chandan Ramanuj



- Applied SMT based attack.
- Reported number of iterations, the CPU time (Time) and the memory usage (RAM) for each test case.

Experimental Results

- Three HLS benchmarks - WAKA, ARF and Motion.

Blue Team

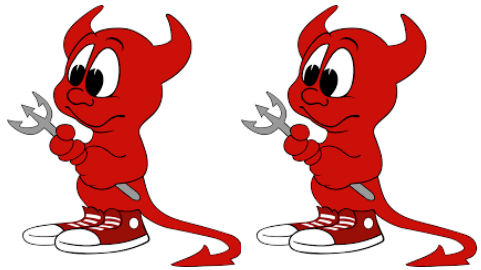
Christian Siddharth Ramesh



- Synthesized these benchmarks with TAO to generate the locked RTL.
- TAO applies operation, control and constant locking.
- Large design with 14K combinational cells and 3K sequential cells.

RedTeam

Chandan Ramanuj



- Applied SMT based attack.
- Reported number of iterations, the CPU time (Time) and the memory usage (RAM) for each test case.
- For successful cases, the time to unlock is under 30 minutes.

Results: Unlocking TAO-locked RTL designs

Bench	LOC	×	+	-	Operations	Conditions	Constants	Key	Comb	Seq	Iterations	Instructions	Time (s)	RAM (MB)
WAKA	753	-	13	7	-	-	3	65	1255	917	4	524	5.16	28
	779	-	23	11	11	4	-	11			5	653	35.46	43
	773	-	23	11	11			9			4	617	92.39	40
	828	-	21	9	9	4	3	73			45	672	1157.13	138
ARF	1431	21	27	10	-	6	-	3	19715	3381	2	6185	517.80	661
	1654	21	27	10	-	-	1	32			2	6863	406.97	576
	1647	21	65	34	65			32			5	6718	>10hrs	-
MOTION	1140	19	11	0	-	-	2	64	13938	2924	5	931	7.01	16
	1239	15	29	10	37	-	-	27			2	885	>10hrs	-
	1250	15	32	10	37	-	4	155			5	924	>10hrs	-

LOC: # of lines in obfuscated Verilog RTL. ×: # of multiplications in Verilog RTL. +: # of adds in Verilog RTL. -: # of subtracts in Verilog RTL. Operations: # of operations obfuscated. Conditions: # of conditions obfuscated. Constants: # of constants obfuscated. Key: # of key bits. Comb: # of combinational cells. Seq: # of sequential cells. Iterations: # of iterations. Instructions: # of instructions executed by KLEE.

Results: Unlocking a locked C code

Bench	Operations	Conditions	Constants	key	Iterations	Instructions	Time (s)	RAM (MB)
WAKA	1	1	5	162	6	306	2888.91	92
	-	-	7	224	8	298	2658.56	120
	2	1	6	195	6	345	3495.51	98
ARF	2	1	1	35	3	1060	1579.77	861
	-	-	4	128	2	1068	400.77	718
	2	1	2	67	3	1142	>10hrs	-
MOTION	2	-	2	66	4	326	11.74	18
	6	-	6	198	8	421	>10hrs	-

Future Scopes and Conclusions

- *Handling Time-outs:* We encountered *time outs* for five instances. We suspect that a time-out implies that no more DIPs exist, i.e., the attack has terminated although Z3 is unable to prove this formally.
- *Limitations:* We did not implement extraction of arrays from Block RAM in RTL. Also, functions in the input C code of TAO are in-lined before RTL generation.
- Presents an SMT attack to recover the secret key from a locked RTL.
- SMT attack abstracts all hardware details into a behavioral program, scaling to large designs.
- Red team unlocked large designs with up to 3K sequential cells and 195 key bits demonstrating the effectiveness of the attack.

Thank you!